“Research Connect”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

PADARTI DIHITHA SAI KRISHNA :2520030015

CHALLA AMRUTHA :2520030412

AMGOTH BHAVIKA PAWAR :2520030616

Under the Guidance of

Dr. V.Sireesha
Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled **"Research Connect"** is a record of Bonafide work of PADARTI DIHITHA SAI KRISHNA – 2520030015, CHALLA AMRUTHA – 2520030412, AMGOTH BHAVIKA PAWAR – 2520030616 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

PADARTI DIHITHA SAI KRISHNA – 2520030015
CHALLA AMRUTHA – 2520030412
AMGOTH BHAVIKA PAWAR – 2520030616

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled **"Research Connect"** is a bonafide work done and submitted by PADARTI DIHITHA SAI KRISHNA – 2520030015, CHALLA AMRUTHA – 2520030412, AMGOTH BHAVIKA PAWAR – 2520030616 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, during the academic year 2026-27.

Signature of the Guide
(Dr. V.Sireesha)

Signature of the HOD
(Dr. P. Venkateswara Rao)

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide, Dr. V. Sireesha, Professor, Department of Computer Science and Engineering (CSE), for her valuable guidance, continuous support, encouragement, and constructive suggestions throughout the development of our project, “Research Connect: Research Paper Search and Recommendation System.” Her expert guidance and insightful feedback helped us gain a deeper understanding of the project and successfully complete our work.

We are highly thankful to Dr. P. Venkateswara Rao, Associate Professor and Head of the Department of Computer Science and Engineering (CSE), for providing us with the necessary facilities, support, and encouragement to carry out this project. His valuable suggestions and motivation played an important role in helping us accomplish our project objectives.

Finally, we would like to thank all the faculty members of the Department of Computer Science and Engineering, our friends, and everyone who directly or indirectly contributed to the successful completion of this project. Their support, cooperation, and encouragement have been invaluable throughout this journey.

PADARTI DIHITHA SAI KRISHNA – 2520030015

CHALLA AMRUTHA – 2520030412

AMGOTH BHAVIKA PAWAR – 2520030616

Abstract

Research papers play a vital role in the advancement of knowledge across various fields. With the rapid growth of scientific publications, researchers and students often face difficulties in finding relevant papers from large collections of academic documents. Traditional search systems mainly depend on exact keyword matching, which may not provide accurate results when queries are incomplete, ambiguous, or contain spelling errors. As a result, there is a need for a more efficient and intelligent research paper discovery system that improves accessibility to academic resources.

Research Connect is a research paper search and recommendation system designed to address these challenges. The system enables users to search research papers using titles, authors, keywords, abstracts, and research domains. To improve search efficiency and accuracy, algorithms such as Knuth-Morris-Pratt (KMP), Rabin-Karp, Z Algorithm, and Edit Distance are utilized. These techniques support both exact and fuzzy matching, allowing users to retrieve relevant papers even when search terms are misspelled or incomplete. The system also includes recommendation features that help users discover related research papers based on similarity and relevance.

In addition to search and recommendation capabilities, Research Connect provides author collaboration analysis, author influence insights and an analytics dashboard to enhance research exploration. The system is implemented using Java for backend processing, HTML, CSS, and JavaScript for the user interface, and file handling techniques for dataset management, and it works on a dataset of 287,421 arXiv papers without using a database. By combining efficient search algorithms, recommendation mechanisms, and analytical features, the proposed system offers a user-friendly platform that improves research discovery, saves time, and supports students, researchers, and academicians in accessing valuable academic information.

Table of Contents

			Page No.
	Acknowledgement		4
	Abstract		5
	Table of Contents		6
	List of Figures		
1	Introduction	1.1 Overview 1.2 Problem Statement 1.3 Scope of the project 1.4 Objectives 1.5 Dataset Used 1.6 Organization of the Report	9-14
2	System Requirements	2.1 Functional Requirements 2.2 Non-Functional Requirements 2.3 Hardware and Software Requirements 2.4 Feasibility Study 2.5 Assumptions and Constraints 2.6 Software Tools Used	15-18
3	System Design	3.1 High-level Architecture Diagram 3.2 Description of each layer 3.3 Data Structures Used 3.4 Use Case Diagram 3.5 Data Flow Diagrams 3.6 Search Request Flow 3.7 Class Diagram 3.8 Algorithm Design 3.9 API Design 3.10 File Design 3.11 Design Decisions 3.12 Algorithm Pseudo-code 3.13 Complexity Analysis 3.14 User Interface Design 3.15 Module Description	19-30
4	Implementation	4.1 Module-wise implementation 4.2 Front-end logic (UI rendering, state management) 4.3 API integration 4.4 Performance considerations 4.5 Testing 4.6 Running the project 4.7 Worked examples of the algorithms 4.8 Additional front-end code 4.9 Complete backend source code 4.10 Front-end source code	31-49
5	Results	5.1 Dataset statistics 5.2 Smart search 5.3 Fuzzy search and "Did you mean" 5.4 Recommendations	50-63

		5.5 Authors and network	
		5.6 Reading list	
		5.7 Analytics dashboard	
		5.8 Comparison of the algorithms	
		5.9 Discussion	
		5.10 Complexity and memory observations	
		5.11 Comparison with plain keyword search	
		5.12 Verification of the functional requirements	
6	Challenges & Limitations	6.1 Issues faced during development 6.2 Solutions applied 6.3 Limitations of the System	64-65
7	Conclusion & Future Work	7.1 Conclusion 7.2 Future Work 7.3 Skills learned during development	66-67
	References	- Books, tutorials, APIs, documentation sites used	68-68

List of Figures

Fig. 1.1 Architecture of Research Connect System....10
Fig. 3.1 Layered architecture of the implemented system....19
Fig. 3.2 Use case diagram of Research Connect....21
Fig. 3.3 Level 0 data flow diagram....21
Fig. 3.4 Level 1 data flow diagram....22
Fig. 3.5 Flow of a search request....23
Fig. 3.6 Sequence diagram of a search....24
Fig. 3.7 Class diagram....24
Fig. 3.8 Flow chart of the dataset loader....25
Fig. 3.9 Flow chart of fuzzy search....25
Fig. 3.10 Flow chart of the recommendation module....26
Fig. 5.1 Home page of Research Connect....50
Fig. 5.2 Suggestions under the search box on the home page....51
Fig. 5.3 Search results for "neural network" using KMP....51
Fig. 5.4 Searching the author field for "Hinton"....52
Fig. 5.5 Fuzzy search for the misspelled query "neural netwrk"....52

Fig. 5.6 "Did you mean" suggestion for "netwrk".....53
Fig. 5.7 Similar papers below a search result.....54
Fig. 5.8 Recommendations page.....54
Fig. 5.9 Author details with collaborators and papers.....55
Fig. 5.10 Top ten authors by influence score.....56
Fig. 5.11 Reading list with CSV text.....56
Fig. 5.12 Analytics dashboard.....57
Fig. 5.13 Papers by domain (top 20).....58
Fig. 5.14 Papers per year.....58
Fig. 5.15 Most frequent title keywords.....59
Fig. 5.16 Number of authors per paper.....60
Fig. 5.17 Best search time of each algorithm.....61

1. INTRODUCTION

1.1 Overview

Research papers are one of the primary sources of knowledge and innovation in academic and scientific communities. With the rapid growth of digital libraries and online repositories, millions of research papers are available across various disciplines. Although this abundance of information is beneficial, it also creates challenges for researchers and students who need to identify relevant papers quickly and efficiently. Traditional search systems mainly depend on exact keyword matching and often fail to provide accurate results when users enter incomplete, misspelled, or ambiguous queries.

Research Connect is a Research Paper Search and Recommendation System designed to improve the process of discovering academic publications. The system enables users to search research papers using titles, authors, keywords, abstracts, and research domains. To enhance search accuracy, efficient pattern-matching algorithms such as Knuth-Morris-Pratt (KMP), Rabin-Karp, Z Algorithm, and Edit Distance are utilized. These algorithms support both exact and fuzzy matching, allowing users to retrieve relevant papers even when search terms are partially entered or contain spelling mistakes.

In addition to search functionality, the system provides recommendation features that suggest related research papers based on content similarity and relevance. Author collaboration analysis and influence scores further help users understand relationships between researchers and identify significant contributors within a research area. The overall architecture of the Research Connect system is shown in Fig. 1.1.

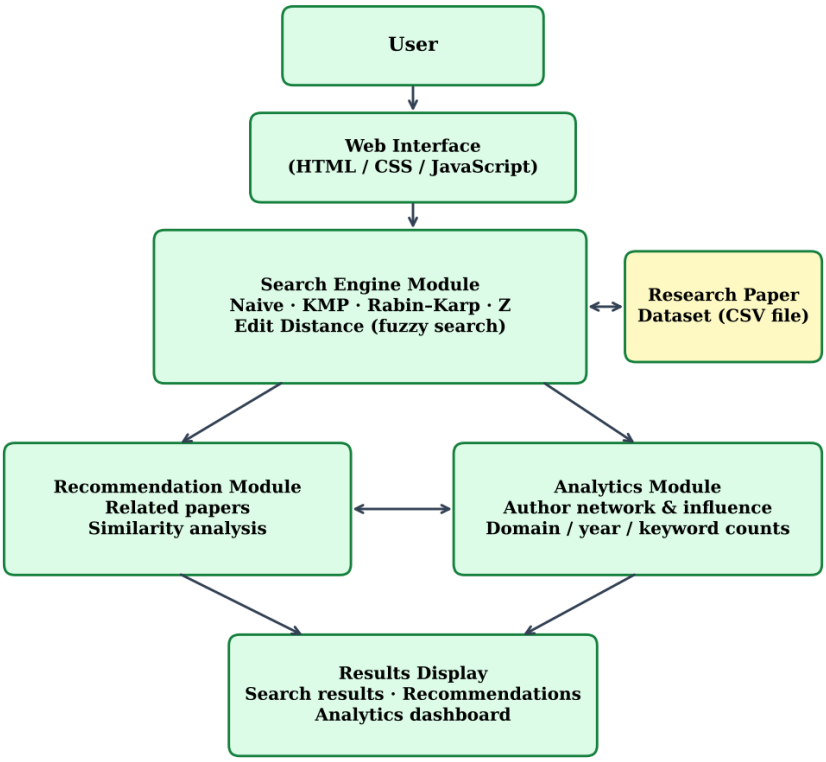


Fig. 1.1 Architecture of Research Connect System

The system is built on a real-world dataset of **287,421 arXiv papers** spread over **146 research domains**, written by **412,677 distinct author names**. It is implemented with Java for backend processing, HTML, CSS and JavaScript for the user interface, and Java file handling for dataset management. No database is used: the dataset is read from a CSV file when the program starts, and the data created by the user (the reading list and the search history) is stored in plain text files.

The project is developed as part of the course on Data Structures and Algorithms. It shows how classical string-matching algorithms, dynamic programming, hash maps, inverted indexes and graphs can be combined to solve a practical information-retrieval problem on a large data set. Each algorithm is implemented by the team in the project itself instead of calling a ready-made library, so that its working can be studied, measured and compared.

1.2 Problem Statement

The rapid growth of digital libraries and online research repositories has resulted in the availability of a vast number of research papers across various academic and scientific disciplines. However, finding relevant research papers from these large collections has become a challenging task for students, researchers, and academicians. Traditional search systems primarily rely on exact keyword matching, which often fails to retrieve appropriate results when users enter incomplete queries, misspelled words, or ambiguous search terms. As a result, users may miss important

publications related to their research interests, leading to increased search time and reduced efficiency in the research discovery process.

To overcome these challenges, Research Connect: Research Paper Search and Recommendation System aims to develop an efficient and user-friendly platform for searching and discovering relevant academic publications. The system utilizes pattern-matching algorithms such as Knuth-Morris-Pratt (KMP), Rabin-Karp, Z Algorithm, and Edit Distance to perform efficient text searching and fuzzy matching across research paper titles, authors, keywords, abstracts, and research domains. By supporting exact and approximate query matching, the system aims to improve search accuracy, handle spelling errors and partial inputs, and provide more relevant results. This approach helps users identify suitable research papers quickly and efficiently, thereby enhancing the overall academic research experience.

The specific problems addressed by the project can be summarized as follows:

- **Information overload:** thousands of papers are published every year, and it is difficult for students and researchers to find the right ones quickly.
- **Poor handling of mistakes:** many search systems cannot handle spelling mistakes or incomplete keywords, which leads to poor search results.
- **Hard discovery of related work:** it is difficult to discover related papers, important authors and new research trends.
- **Scattered information:** research information is spread across different sources, which makes it difficult to search, organize and analyze it in one place.

1.3 Scope of the project

The scope of the Research Connect: Research Paper Search and Recommendation System is to develop an efficient and user-friendly platform that helps students, researchers, and academicians search for and discover relevant research papers easily. The system allows users to search for academic publications based on titles, authors, keywords, abstracts, and research domains. It aims to reduce the time and effort required to identify useful research papers from large collections of academic documents.

The system incorporates efficient string-matching algorithms, including Knuth-Morris-Pratt (KMP), Rabin-Karp, and the Z Algorithm, to support fast and accurate text searching. Edit Distance is used to identify approximate matches and handle spelling errors, incomplete inputs, and variations in search terms. These techniques improve the flexibility and effectiveness of the search process compared with traditional keyword-based systems.

Furthermore, Research Connect aims to improve research discovery by presenting relevant search results based on the user's query. The system can be extended in the future by integrating online academic databases, advanced recommendation techniques, citation analysis, user preferences, and machine learning methods. Overall, the project focuses on making academic paper discovery faster, more accurate, and more accessible while providing a foundation for further improvements in research paper search and recommendation.

Within this scope, the implemented version provides the following capabilities:

- Searching papers by title, author, abstract or all fields, with an optional research-domain filter and three sort orders.
- Choosing the pattern-matching algorithm (Naive, KMP, Rabin-Karp or Z) and a fuzzy mode based on Edit Distance.
- Search suggestions while typing and "Did you mean" corrections for misspelled words.
- Recommendations of related papers from the papers the user saves or opens.
- An author collaboration network with collaborator lists and an influence score.
- An analytics dashboard with papers per domain and per year, top authors and trending keywords.
- A reading list with CSV export and a search history, both kept in text files through Java file handling.

The following are outside the scope of this version: real citation counts and citation links (the arXiv dataset used does not contain citation information), user accounts and logins, and downloading the full text of papers.

1.4 Objectives

The main objective of Research Connect: Research Paper Search and Recommendation System is to develop an efficient and user-friendly platform that enables users to search, identify, and discover relevant research papers quickly and accurately. The specific objectives of the project are:

- **To develop an efficient search system:** Provide a platform to search research papers using titles, authors, keywords, abstracts, and research domains.
- **To implement pattern-matching algorithms:** Utilize Knuth-Morris-Pratt (KMP), Rabin-Karp, and the Z Algorithm for efficient text searching and pattern identification.
- **To support fuzzy searching:** Use the Edit Distance algorithm to handle spelling mistakes, incomplete queries, and variations in search terms.

- **To improve search accuracy:** Retrieve relevant research papers even when users enter partial or incorrectly spelled keywords.
- **To reduce search time:** Minimize the time and effort required to find suitable academic publications from a collection of research papers.
- **To provide relevant search results:** Display research papers that closely match the user's search query, making the research discovery process more effective.
- **To enhance user accessibility:** Design a simple and user-friendly interface that allows students, researchers, and academicians to search for research papers conveniently.
- **To support future enhancements:** Provide a foundation for integrating academic databases, advanced recommendation methods, citation analysis, and machine learning techniques in the future.

1.5 Dataset Used

The papers come from the public arXiv scientific dataset, supplied as one CSV file of about 430 MB. Every row describes one paper. The fields used by the system are listed in Table 1.1. The abstracts and titles contain commas, quotation marks and line breaks inside quoted fields, so the file cannot be read by simply splitting lines at commas; the loader described in Chapter 4 handles this.

Table 1.1 Dataset fields used by the system

Field	Meaning	Used for
id	arXiv identifier of the paper	Unique key, saving papers
title	Title of the paper	Search, suggestions, recommendations, keyword counts
category	Research domain	Domain filter, analytics
published date	Date of publication (two-digit year)	Year, sorting, analytics
authors	List of author names	Author search, network, influence
summary	Abstract of the paper	Abstract search, display of the first 400 characters

When the full file is loaded the program reports 287,421 papers, 146 research domains and 412,677 distinct author names. On average a title has 9.6 words and an abstract has 171 words.

1.6 Organization of the Report

The rest of the report is organized as follows. Chapter 2 lists the hardware, software and functional requirements. Chapter 3 describes the design of the system with its architecture, data structures,

diagrams and file design. Chapter 4 explains the implementation of every module with code. Chapter 5 presents the results with screenshots, measurements and a comparison of the algorithms. Chapter 6 discusses the challenges faced, the solutions applied and the limitations. Chapter 7 gives the conclusion, future work and the skills learned. The references are listed at the end.

2. SYSTEM REQUIREMENTS

Research Connect runs on an ordinary laptop and requires Java Development Kit (JDK) 17 or newer to execute the application. It supports Windows, macOS, and Linux operating systems and can be accessed through modern web browsers such as Google Chrome, Microsoft Edge, Mozilla Firefox, or Safari. Visual Studio Code with the Extension Pack for Java can be used as an optional development environment. The system does not require an external database, additional libraries, or an internet connection because it runs locally at localhost:8080. The minimum hardware requirement is 4 GB of RAM, while 8 GB is recommended for comfortable performance. Approximately 430 MB of disk space is required for the dataset containing 287,421 research papers, along with a small amount of additional space for saved search results and history files. The application uses up to 3 GB of memory to load the dataset into RAM. On a system with 4 GB RAM and a single CPU core, dataset loading takes approximately 34 seconds, exact searches using Naive, KMP, Rabin-Karp, or Z algorithms take around 0.8-2 seconds, and fuzzy searches take around 4-7 seconds. Search performance may improve with a multi-core processor. These requirements enable efficient searching and retrieval of research papers using exact and fuzzy pattern-matching techniques.

2.1 Functional Requirements

The functional requirements of Research Connect: Research Paper Search and Recommendation System describe the main operations and services provided by the application. The system should load and process a dataset containing 287,421 research papers and store the paper records in memory for efficient searching. It should allow users to search research papers using titles, authors, keywords, abstracts, and other available text fields. The system should support exact search using the Naive, Knuth-Morris-Pratt (KMP), Rabin-Karp, and Z algorithms, along with fuzzy search using Edit Distance to handle spelling mistakes and approximate matches. It should display relevant search results through a web interface accessible at localhost:8080. The application should also allow users to save selected papers and maintain search history using local text files such as saved.txt and history.txt. Furthermore, the system should support parallel searching where applicable, provide appropriate results for valid queries, and allow users to access the application through a modern web browser without requiring an internet connection or an external database.

The functional requirements are listed individually in Table 2.1, and each of them is verified in the testing section of Chapter 4.

Table 2.1 Functional requirements

ID	Requirement
FR1	The user can search papers by title, author, abstract or all fields.
FR2	The user can filter results by research domain and sort them by relevance, newest or oldest.
FR3	The user can choose the algorithm: Naive, KMP, Rabin-Karp, Z or Fuzzy (Edit Distance).
FR4	The system tolerates spelling mistakes and suggests corrections ("Did you mean").
FR5	The system suggests keywords, paper titles and authors while the user types.
FR6	The system recommends papers related to the papers the user saved or opened.
FR7	The user can open an author to see the papers, collaborators and influence score.
FR8	The system shows analytics: papers per domain and year, top authors and trending keywords.
FR9	The user can save papers to a reading list and view it as CSV text.
FR10	The system remembers recent searches and lets the user run them again.
FR11	The system shows the number of matches and the search time after every search.
FR12	Results are delivered 15 at a time with a Show more button.

2.2 Non-Functional Requirements

- **Performance:** the KMP, Rabin-Karp and Z algorithms run in linear time, and the scan over all papers is done in parallel on all available CPU cores.
- **Usability:** a clean white and green interface with navigation buttons at the top, suggestions while typing and results shown 15 at a time.
- **Scalability:** results are paginated, so the browser never has to hold all 287,421 papers.
- **Reliability:** quoted CSV fields that span several lines are parsed correctly, and the user data survives restarts because it is stored in files.
- **Portability:** only the Java Development Kit and a web browser are needed; no database or extra library is installed.
- **Maintainability:** one Java source file for the backend and two small files for the front end, with the modules clearly separated.
- **Security:** all text shown in the browser is HTML-escaped, and the web server only serves

files from the web folder.

- **Accuracy:** all four exact algorithms must return the same set of papers for the same query; they may differ only in speed.

2.3 Hardware and Software Requirements

Table 2.2 Hardware and software requirements

Item	Requirement
Processor	Any multi-core CPU (the search uses parallel streams); a single core also works
Memory (RAM)	4 GB minimum, 8 GB recommended; about 3 GB is given to the Java program (-Xmx3g)
Disk space	About 500 MB (the CSV file is about 430 MB)
Operating system	Windows, macOS or Linux
Backend	Java Development Kit 17 or newer; built-in com.sun.net.httpserver
Frontend	HTML5, CSS3 and JavaScript (ES6) in any modern browser
Editor (optional)	Visual Studio Code with the Extension Pack for Java
Network	Not needed; the application runs on localhost:8080
Database	None

2.4 Feasibility Study

2.4.1 Technical feasibility

All the technology used is freely available and works on common computers. Java is installed on most student systems, the HTTP server is part of the JDK, and the browser is used for the interface. The only demanding part is the memory needed to keep the whole dataset in RAM, which was tested on a computer with 4 GB of memory.

2.4.2 Economic feasibility

The project needs no paid software, licence, hosting or database server. The JDK, Visual Studio Code and the arXiv dataset are free, so the cost of developing and running the system is practically zero.

2.4.3 Operational feasibility

The user only starts the program once and opens a web address. The interface follows the familiar pattern of a search box with filters and result cards, so no training is needed. Reading list and history are saved automatically.

2.5 Assumptions and Constraints

- The dataset file is available in the data folder before the program is started, and it keeps the column order of the arXiv file that was supplied.
- The user runs the program on a personal computer; the application is not meant to be used by many people at the same time.
- The user enters queries in English, because the title words are split with the pattern [^a-z] and the dataset is in English.
- The computer has enough free memory for the Java heap given with -Xmx3g; with less memory a smaller sample of the CSV must be used.
- Port 8080 is free; otherwise another port can be given as the second argument of the program.
- Two papers are treated as the same author if the author names are written identically, because the dataset has no author identifiers.

2.6 Software Tools Used

Table 2.3 Tools used in the development

Tool	Use
Java Development Kit	Compiling and running the backend
Visual Studio Code	Editing the code; Extension Pack for Java for running and debugging
Web browser (Chrome)	Opening the interface and checking the front end
Notepad / text editor	Viewing saved.txt and history.txt
Python and matplotlib	Calculating dataset statistics and drawing the diagrams and charts of this report
Microsoft Word	Writing this report

3. SYSTEM DESIGN

3.1 High-level Architecture Diagram

Research Connect follows a layered client-server design. The browser sends requests to a Java server running on the same computer. The server keeps the whole dataset in memory, applies the search, recommendation and analytics modules, and sends the answer back as JSON. Fig. 3.1 shows the architecture of the implemented system.

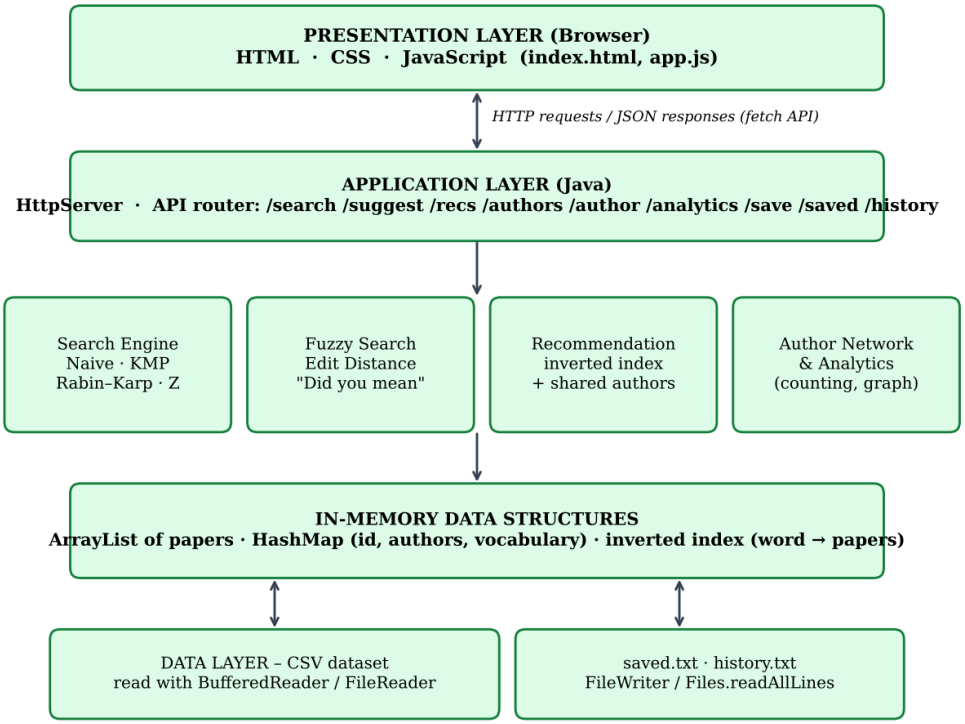


Fig. 3.1 Layered architecture of the implemented system

3.2 Description of each layer

3.2.1 Presentation layer

The presentation layer is a single-page web application made of index.html (structure and styling) and app.js (behaviour). It has six sections: Home, Search, Recommendations, Authors & Network, Analytics and Reading List. The navigation bar at the top shows one section and hides the others. The page never searches the data itself; it only asks the server for results and draws the answer.

3.2.2 Application layer

The application layer is the Java program ResearchConnect.java. It uses the HTTP server that is built into the JDK. One handler receives every request: addresses that start with /api/ are routed to the matching feature and answered with JSON, and every other address is answered with a file from the web folder. A fixed pool of four threads serves requests, and the search itself uses a parallel

stream.

3.2.3 Processing modules

- **Search engine:** the Naive, KMP, Rabin-Karp and Z string-matching algorithms.
- **Fuzzy search:** Edit Distance (dynamic programming) to find near spellings and corrections.
- **Recommendation module:** an inverted index of title words plus shared-author and same-domain bonuses.
- **Author network and analytics:** author-to-papers maps, collaborator counting and frequency counting.

3.2.4 Data layer

At start-up the CSV file is read once with BufferedReader and converted into Java objects that are held in memory. The data created by the user, the reading list and the search history, is kept in two text files inside the data folder and is read and written with Java file handling. No database is involved.

3.3 Data Structures Used

The choice of data structures decides how fast each feature runs. Table 3.1 lists the main structures; all of them are built once while the CSV is being read.

Table 3.1 Main data structures

Structure	Holds	Why it is used
ArrayList<Paper>	All papers in file order	Fast access by position and easy to scan in parallel
HashMap id to index	Paper id and its position	Constant-time lookup when saving or recommending
HashMap author to papers	Each author with the positions of their papers	Author pages and collaborator counting
HashMap word to count	Vocabulary of title words with frequencies	Suggestions, fuzzy search, trending keywords
HashMap word to papers	Inverted index of title words	Finds similar papers without scanning every paper
TreeMap domain to count	Papers per domain, kept sorted by name	Domain list and analytics

The inverted index is the same idea that is used inside real search engines: instead of looking through all papers for a word, the program looks the word up and directly gets the list of papers that contain it.

3.4 Use Case Diagram

The only actor is the user, a student or researcher. Fig. 3.2 shows the use cases that the system offers. All of them are available from the web interface without any login.

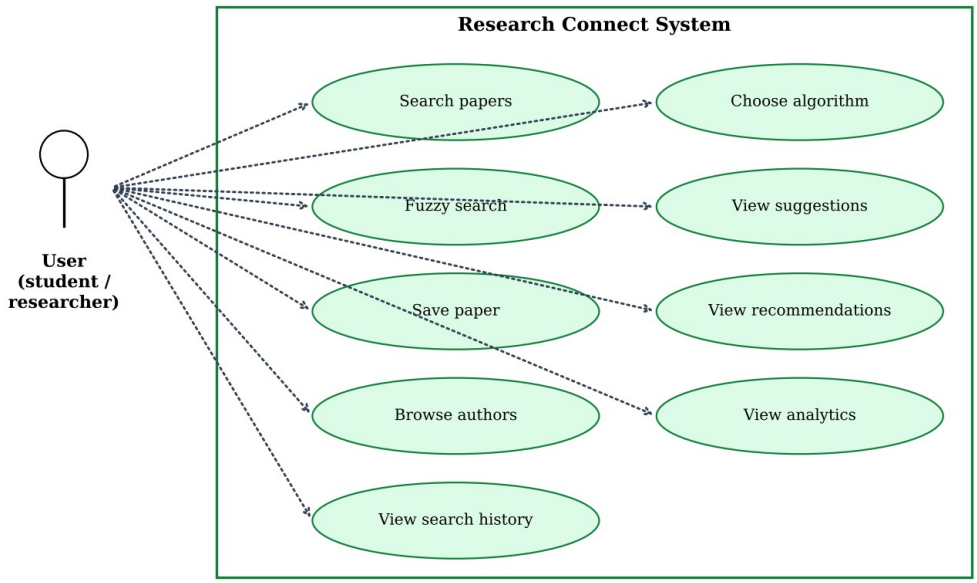


Fig. 3.2 Use case diagram of Research Connect

3.5 Data Flow Diagrams

Fig. 3.3 is the context-level data flow diagram. The user sends queries and clicks to the system and gets results and charts back. The system reads papers from the CSV dataset and reads and writes the saved list and the history in text files.

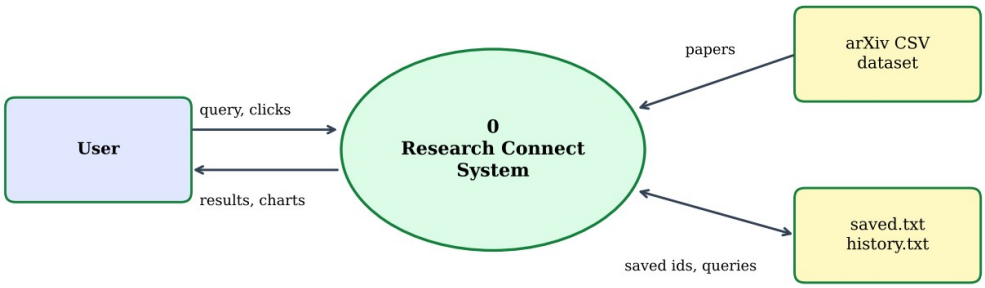


Fig. 3.3 Level 0 data flow diagram

Fig. 3.4 splits the system into seven processes. Process 1 loads the dataset into the in-memory structures (D2). Processes 2 to 6 read from D2 to search, suggest, recommend, build the author network and compute analytics. Process 7 reads and writes the user files (D3).

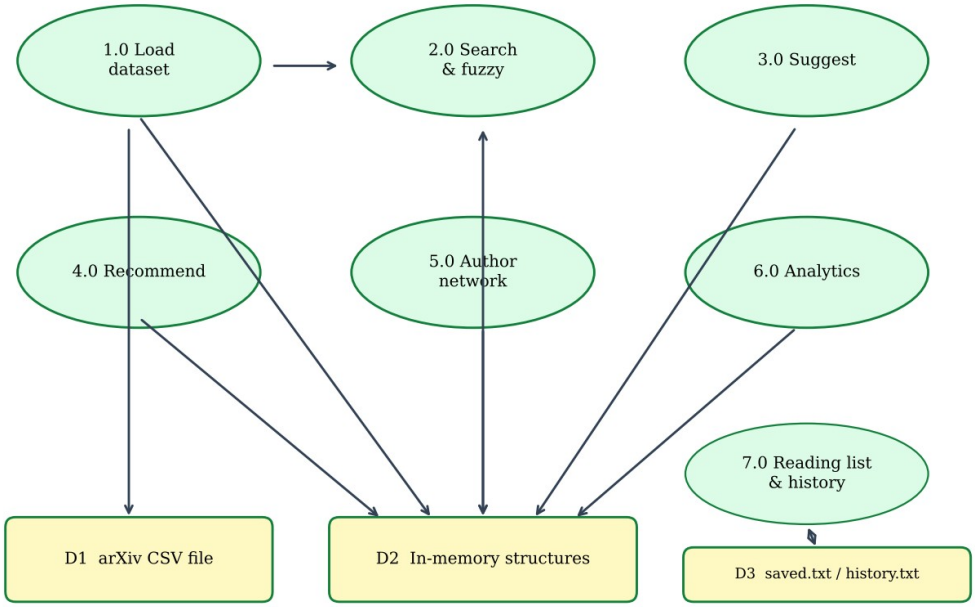


Fig. 3.4 Level 1 data flow diagram

3.6 Search Request Flow

Fig. 3.5 shows what happens from the moment the user presses Search until the results are displayed. Only 15 results are sent at a time; the Show more button asks for the next 15 using an offset.

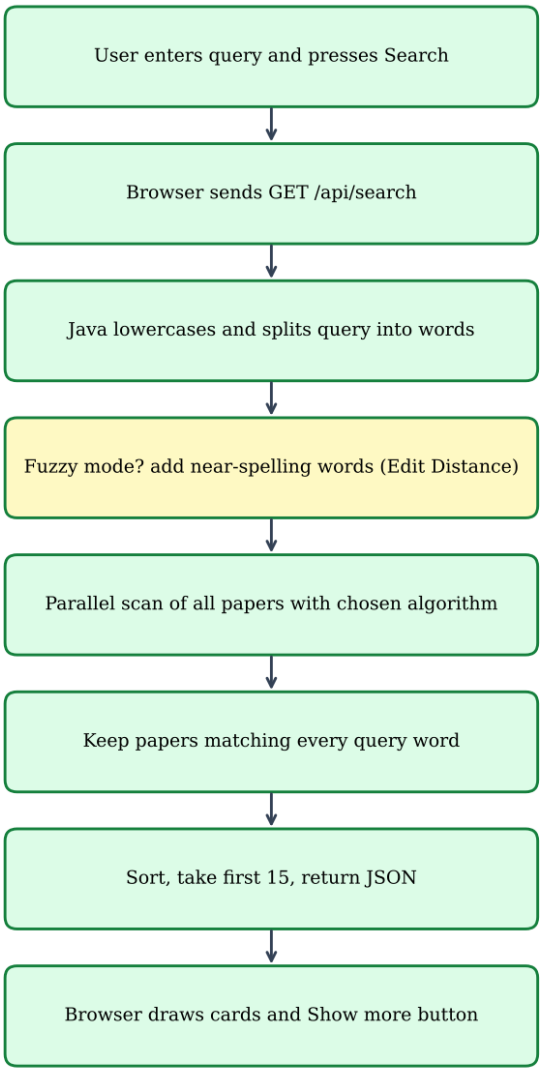


Fig. 3.5 Flow of a search request

The same interaction is shown as a sequence diagram in Fig. 3.6. The browser talks only to the HTTP server; the server calls the search module, which scans the in-memory papers, and finally the query is appended to the history file.

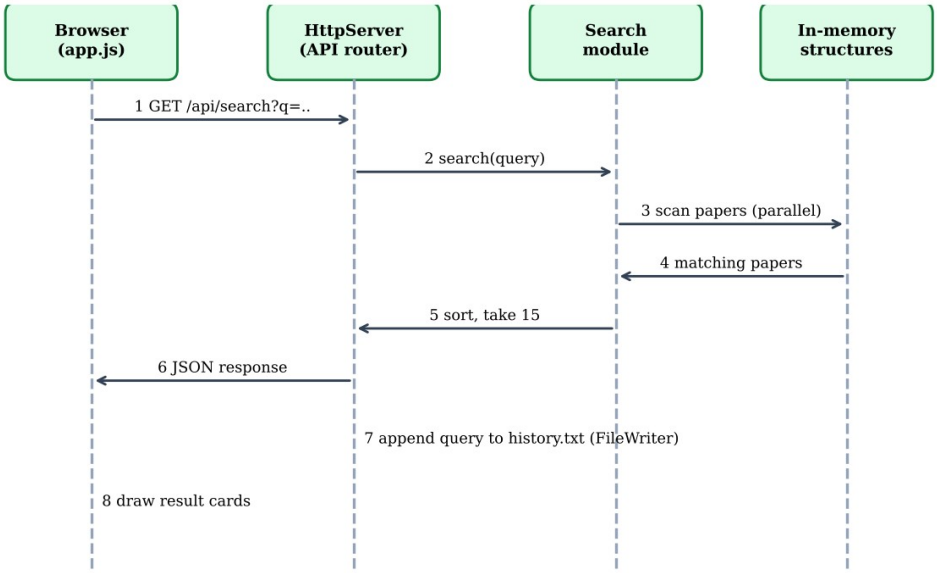


Fig. 3.6 Sequence diagram of a search

3.7 Class Diagram

The program is deliberately small. A Paper class holds the data of one paper, and the main class ResearchConnect keeps the static collections and all the methods. The HTTP server of the JDK is used to receive requests. Fig. 3.7 shows the classes and their main members.

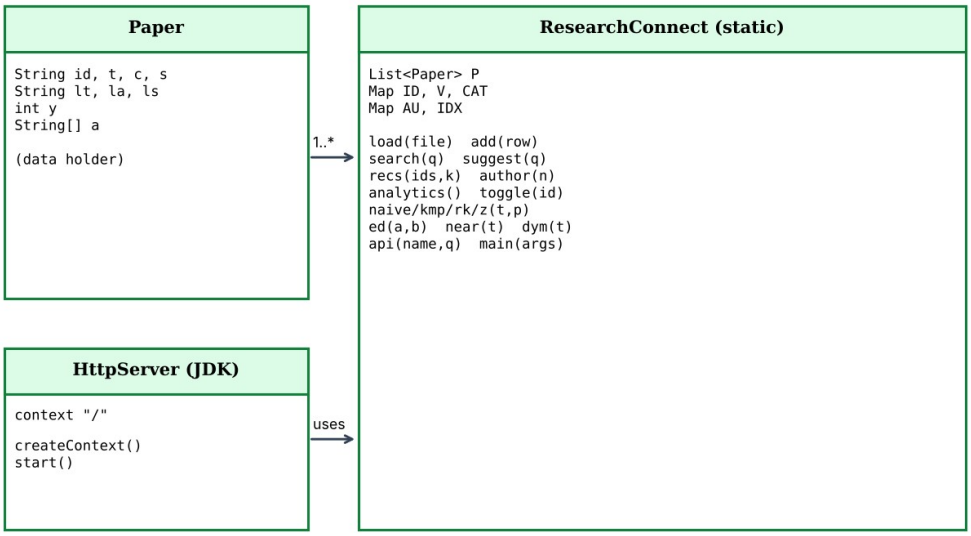


Fig. 3.7 Class diagram

3.8 Algorithm Design

The loading, the fuzzy search and the recommendation module are the three parts that are not a single textbook algorithm but a sequence of steps. Their flow charts are given in Fig. 3.8 to Fig. 3.10.

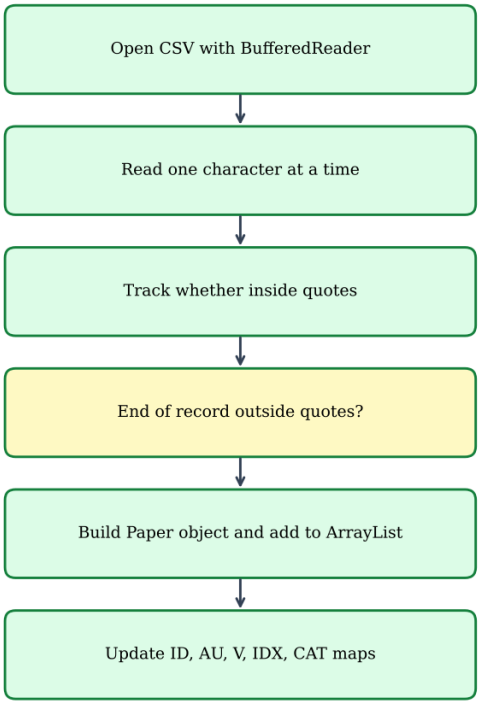


Fig. 3.8 Flow chart of the dataset loader

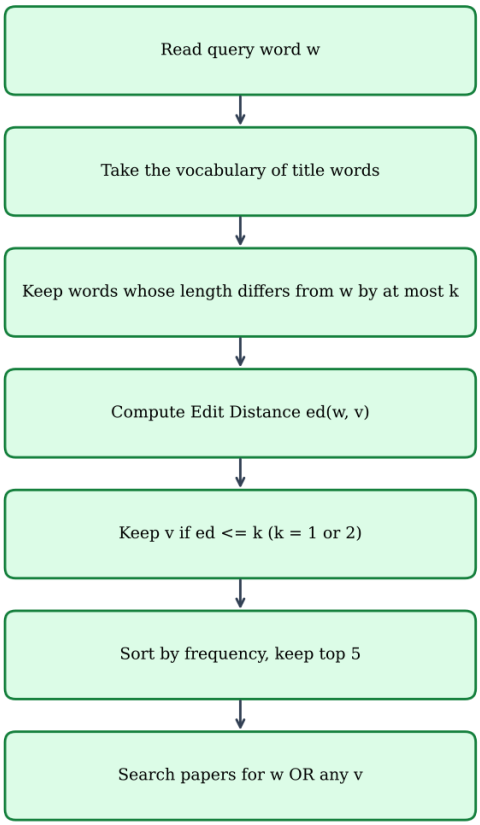


Fig. 3.9 Flow chart of fuzzy search

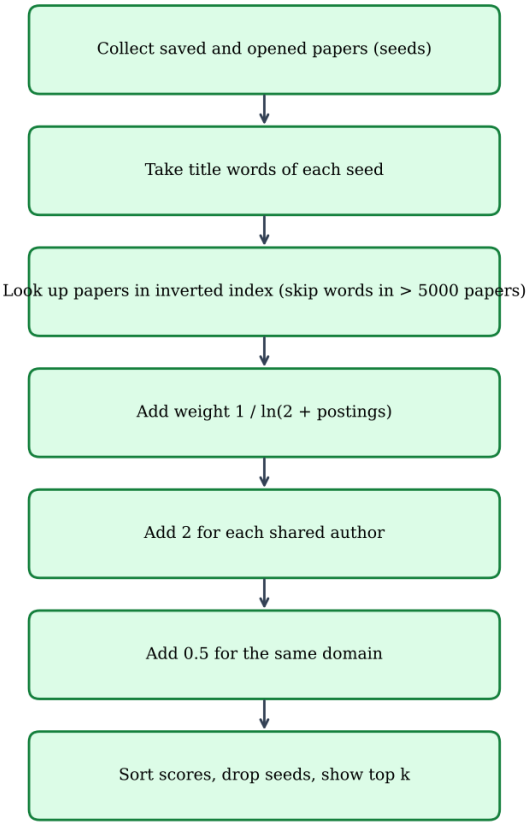


Fig. 3.10 Flow chart of the recommendation module

3.9 API Design

The browser and the Java program communicate through a small REST-style interface. Every endpoint is a GET request that returns JSON. The endpoints are listed in Table 3.2.

Table 3.2 Endpoints of the Java server

Endpoint	Purpose
/api/meta	Total papers, number of authors and the list of domains
/api/search?q&field&cat&alg&sort&offset	Search; returns the total, the time taken and 15 papers
/api/suggest?q	Keyword, paper title and author suggestions
/api/recs?ids&k	Recommended papers for the given paper ids
/api/authors?q	List of authors (top authors when q is empty)
/api/author?name	Papers, collaborators and influence of one author
/api/analytics	Counts for the dashboard
/api/save?id	Add or remove a paper in the reading list
/api/saved	List of saved papers
/api/history	The last six distinct searches

3.10 File Design

Three files are used. The dataset CSV is only read. The file saved.txt stores one paper id per line for the reading list, and history.txt stores one query per line. Both are created automatically in the data folder. Table 3.3 summarizes them.

Table 3.3 Files used by the system

File	Access	Content
data/ arXiv_scientific_dataset.csv	Read with BufferedReader	The 287,421 papers
data/saved.txt	Read with Files.readAllLines, written with Files.write	One paper id per line (reading list)
data/history.txt	Appended with FileWriter	One query per line (search history)

3.11 Design Decisions

- **No database:** the project has to use Java file handling, and the data is read only, so a database would add complexity without a benefit.
- **Everything in memory:** keeping all papers in RAM makes every search a memory scan instead of a disk operation; the price is the memory use of about 3 GB.
- **JDK web server instead of Spring Boot:** nothing needs to be downloaded or configured, and the whole backend is one file that is easy to study.
- **Pagination:** the server sends 15 papers at a time so that the response stays small however many papers match.
- **Own implementation of algorithms:** KMP, Rabin-Karp, Z, Naive and Edit Distance are written in the project so their behaviour can be measured and explained.

3.12 Algorithm Pseudo-code

The pseudo-code of the four algorithms and of the Edit Distance calculation is given below. It is independent of the programming language and corresponds to the Java methods of Chapter 4.

3.12.1 Naive search

```
NAIVE(text T, pattern P)
  for i = 0 to |T| - |P|
      j = 0
      while j < |P| and T[i + j] == P[j]
          j = j + 1
      if j == |P| then return FOUND
  return NOT FOUND
```

3.12.2 Knuth-Morris-Pratt

```
BUILD-FAILURE(P)
  f[0] = 0 ; k = 0
  for i = 1 to |P| - 1
      while k > 0 and P[i] != P[k]  do  k = f[k - 1]
      if P[i] == P[k] then k = k + 1
      f[i] = k
  return f

KMP(T, P)
  f = BUILD-FAILURE(P) ; k = 0
  for i = 0 to |T| - 1
      while k > 0 and T[i] != P[k]  do  k = f[k - 1]
      if T[i] == P[k] then k = k + 1
      if k == |P| then return FOUND
  return NOT FOUND
```

3.12.3 Rabin-Karp

```
RABIN-KARP(T, P)          // base B = 256, modulus M = 1000003
  hp = hash(P) ; ht = hash(T[0 .. |P|-1]) ; pw = B^(|P|-1) mod M
  for i = 0 to |T| - |P|
      if hp == ht and T[i .. i+|P|-1] == P then return FOUND
      if i + |P| < |T| then
          ht = ((ht - T[i] * pw) * B + T[i + |P|]) mod M
  return NOT FOUND
```

3.12.4 Z Algorithm

```
Z-SEARCH(T, P)
  S = P + "#" + T            // "#" does not occur in the text
  l = 0 ; r = 0
  for i = 1 to |S| - 1
      if i < r then z[i] = min(r - i, z[i - l])
      while i + z[i] < |S| and S[z[i]] == S[i + z[i]]  do  z[i] = z[i] + 1
      if i + z[i] > r then  l = i ; r = i + z[i]
      if z[i] == |P| then return FOUND
  return NOT FOUND
```

3.12.5 Edit Distance

```
EDIT-DISTANCE(a, b)
  d[j] = j  for j = 0 .. |b|
  for i = 1 to |a|
      prev = d[0] ; d[0] = i
      for j = 1 to |b|
          tmp = d[j]
          d[j] = min( d[j] + 1,            // deletion
                      d[j - 1] + 1,        // insertion
                      prev + (a[i] != b[j]) ) // substitution
          prev = tmp
  return d[|b|]
```

3.13 Complexity Analysis

Let n be the length of the text that is searched, m the length of the pattern, N the number of papers, V the number of words in the vocabulary and L the length of a word. Table 3.4 gives the time and the extra memory of the algorithms.

Table 3.4 Time and space complexity

Algorithm	Time	Extra space	Remark
Naive	O(n x m) worst, O(n) usual	O(1)	No preprocessing
KMP	O(n + m)	O(m)	Failure table of the pattern
Rabin-Karp	O(n + m) average, O(n x m) worst	O(1)	Worst case when many hashes collide
Z Algorithm	O(n + m)	O(n + m)	Z-array of the combined string
Edit Distance	O(L1 x L2)	O(L2)	One row of the table is kept
Fuzzy search (vocabulary scan)	O(V x L1 x L2)	O(V)	Done once per query word
Exact search of all papers	O(N x (n + m)) divided by the number of cores	O(1) per paper	Parallel stream

The memory used by the program is dominated by the 287,421 papers with their lowercase copies of title, authors and abstract, which is the reason for the heap size of 3 GB. The inverted index and the author map need additional memory in proportion to the number of words and authors.

3.14 User Interface Design

The interface follows a simple and professional design with a white background and green highlights. The top navigation bar is fixed to the top of the window and contains the logo and six buttons. The active button is filled with green so that the user always knows the current page. Paper cards and boxes use a light green fill with a thin green border, and links and charts use a darker green. A responsive layout makes the cards stack on narrow screens.

Table 3.5 Pages of the interface

Page	Content
Home	Heading, search box with suggestions, three statistic boxes and the feature cards
Search	Search box, field, domain, algorithm and sort selectors, recent searches, result count and time, paper cards
Recommendations	Papers related to the saved and opened papers with match percentage
Authors & Network	Filter box, list of authors and the detail panel of the selected author
Analytics	Bar charts of domains, top authors, years and keywords
Reading List	Saved papers and the CSV text box

3.15 Module Description

Table 3.6 Modules of Research Connect

Module	Input	Processing	Output
Dataset loader	CSV file	Quote-aware reading, conversion of year, building of the maps and the inverted index	Papers and lookup structures in memory
Search engine	Query, field, domain, algorithm, sort, offset	Parallel scan with Naive, KMP, Rabin-Karp or Z; sorting; pagination	Total, time and 15 papers
Fuzzy search	Misspelled query word	Edit Distance against the vocabulary; selection of five alternatives	Alternative words and "did you mean" word
Suggestion module	Typed text	Prefix match on vocabulary, substring match on titles and authors	Up to twelve labelled suggestions
Recommendation module	Saved and opened paper ids	Weighted inverted-index lookup plus author and domain bonuses	Related papers with match percentage
Author module	Author name	Collaborator counting from the author map	Papers, collaborators, influence
Analytics module	None	Counting and sorting of domains, years, authors and keywords	Data of four charts
Persistence module	Paper id or query	File reading and writing with Files and FileWriter	saved.txt and history.txt

4. IMPLEMENTATION

4.1 Module-wise implementation

The backend is a single Java file, ResearchConnect.java, and the front end consists of index.html and app.js. This section explains each module with the code of the project. The code is shown in a shortened form; long lines are wrapped.

4.1.1 Dataset loader (file handling)

The CSV file cannot be split at commas or line breaks because the abstracts contain both inside quotation marks. The loader therefore reads the file one character at a time with BufferedReader and remembers whether it is inside quotes. A comma or a line break ends a field or a record only when it is outside quotes. Every finished record is passed to the method add(), which converts it into a Paper object and updates all the lookup structures of Table 3.1.

```
    static void load(String file) throws IOException {
        try (BufferedReader br = new BufferedReader(new FileReader(file, StandardCharsets.UTF_8),
1
            << 16)) {
            br.readLine();
            StringBuilder cur = new StringBuilder(); List<String> f = new ArrayList<>(); boolean
q =
                false; int c;
            while ((c = br.read()) != -1) {
                if (c == '"') q = !q;
                else if (c == ',' && !q) { f.add(cur.toString()); cur.setLength(0); }
                else if (c == '\n' && !q) { f.add(cur.toString()); cur.setLength(0); add(f);
                    f.clear(); }
                else if (c != '\r') cur.append((char) c);
            }
        }
    }
```

The method add() converts the two-digit year of the publication date into four digits (values above 26 are read as 19xx, the others as 20xx), keeps lowercase copies of the title, authors and abstract so that searching ignores letter case, stores only the first 400 characters of the abstract for display, and fills the maps. The authors of one paper are added to the author map only once, so that an author who is listed twice in a paper is not counted twice.

```
    static void add(List<String> f) {
        if (f.size() < 9) return;
        Paper p = new Paper(); p.id = f.get(0); p.t = f.get(1).replaceAll("\\s+", " ").trim();
p.c =
            f.get(2);
        String[] d = f.get(4).split("/"); int y;
        try { y = Integer.parseInt(d[d.length - 1].trim()); } catch (Exception ex) { return; }
        p.y = y < 100 ? (y > 26 ? 1900 + y : 2000 + y) : y;
        p.a = parseAuthors(f.get(6));
        String s = f.get(8).replaceAll("\\s+", " ").trim();
        p.ls = s.toLowerCase(); p.s = s.length() > 400 ? s.substring(0, 400) : s;
        p.lt = p.t.toLowerCase(); p.la = String.join(" ", p.a).toLowerCase();
        int i = P.size(); P.add(p); ID.put(p.id, i); CAT.merge(p.c, 1, Integer::sum);
        for (String a : new LinkedHashSet<>(Arrays.asList(p.a))) if (!a.isEmpty())
            AU.computeIfAbsent(a, k -> new ArrayList<>()).add(i);
        for (String w : new HashSet<>(Arrays.asList(p.lt.split("[^a-z]+"))))
            if (w.length() >= 3) { V.merge(w, 1, Integer::sum); if (w.length() >= 4 &&
                !STOP.contains(w)) IDX.computeIfAbsent(w, k -> new ArrayList<>()).add(i); }
    }
```

The author column is a list written as text, for example ['A. Author', 'B. Author']. The method parseAuthors() removes the brackets and quotes and splits the names.

```
    static String[] parseAuthors(String s) {
        s = s.trim();
        if (s.startsWith("[") && s.endsWith("]")) s = s.substring(1, s.length() - 1);
        String[] r = s.split("['\"],\\s*['\"]");
        for (int i = 0; i < r.length; i++) r[i] = r[i].replaceAll("^['\"]|['\"]$", "").trim();
        return r;
    }
```

4.1.2 Search engine and the four pattern-matching algorithms

A paper matches a query when every word of the query is found in the chosen field (title, authors, abstract or all three). The word is looked up in the text with the algorithm selected by the user. All four algorithms give the same answer and differ only in how they work, as summarized in Table 4.1.

Table 4.1 Pattern-matching algorithms used

Algorithm	Main idea	Time
Naive	Compare the pattern at every position, letter by letter	O(n x m) worst case
KMP	Build a failure table of the pattern (longest prefix that is also a suffix) so the text is never read twice	O(n + m)
Rabin-Karp	Compare a rolling hash of each text window with the hash of the pattern; compare letters only when the hashes are equal	O(n + m) average
Z Algorithm	Build pattern + separator + text and compute the Z-array; a value equal to the pattern length is a match	O(n + m)

Naive search. For every start position the pattern is compared letter by letter with the text. It is the simplest method and the baseline for comparison.

```
    static boolean naive(String t, String p) {
        for (int i = 0; i <= t.length() - p.length(); i++) { int j = 0; while (j < p.length() &&
            t.charAt(i + j) == p.charAt(j)) j++; if (j == p.length()) return true; }
        return false;
    }
```

Knuth-Morris-Pratt (KMP). The failure table f[i] stores the length of the longest proper prefix of the pattern that is also a suffix of its first i+1 letters. When a mismatch happens, the pattern is shifted using this table, so the text pointer never moves backwards.

```
    static boolean kmp(String t, String p) {
        int[] f = new int[p.length()];
        for (int i = 1, k = 0; i < p.length(); i++) { while (k > 0 && p.charAt(i) != p.charAt(k))
k
            = f[k - 1]; if (p.charAt(i) == p.charAt(k)) k++; f[i] = k; }
        for (int i = 0, k = 0; i < t.length(); i++) { while (k > 0 && t.charAt(i) != p.charAt(k))
k
            = f[k - 1]; if (t.charAt(i) == p.charAt(k)) k++; if (k == p.length()) return true; }
        return false;
    }
```

Rabin-Karp. The hash of the pattern and the hash of the first text window are calculated with base 256 and modulus 1000003. The window then slides one letter at a time and its hash is updated in constant time by removing the first letter and adding the next one. Only when the two hashes are equal are the letters compared, which removes the false positives caused by hash collisions.

```
    static boolean rk(String t, String p) {
        int m = p.length(), n = t.length(); if (m > n) return false;
        final long M = 1000003; long h = 0, w = 0, pw = 1;
        for (int i = 0; i < m; i++) { h = (h * 256 + p.charAt(i)) % M; w = (w * 256 +
t.charAt(i)) %
            M; if (i > 0) pw = pw * 256 % M; }
        for (int i = 0; ; i++) {
            if (h == w && t.startsWith(p, i)) return true;
            if (i + m >= n) return false;
            w = ((w - t.charAt(i) * pw % M + M) * 256 + t.charAt(i + m)) % M;
        }
    }
```

Z Algorithm. The string pattern + separator + text is built, where the separator is a character that cannot occur in the data. The Z-array stores, for every position, the length of the longest substring that starts there and matches a prefix of the whole string. A position whose value equals the length of the pattern is a match.

```
    static boolean z(String t, String p) {
        String s = p + "\u0001" + t; int n = s.length(), m = p.length(); int[] z = new int[n];
        for (int i = 1, l = 0, r = 0; i < n; i++) {
            if (i < r) z[i] = Math.min(r - i, z[i - l]);
            while (i + z[i] < n && s.charAt(z[i]) == s.charAt(i + z[i])) z[i]++;
            if (i + z[i] > r) { l = i; r = i + z[i]; }
            if (z[i] == m) return true;
        }
        return false;
    }
```

The method m() selects the algorithm requested by the browser, and search() combines everything: it lowercases the query, splits it into words, builds the list of alternative spellings when the fuzzy mode is on, and then scans all papers in parallel with IntStream.range(...).parallel(). A paper is kept

when, for every query word, at least one of its alternatives is found in one of the selected fields. The result is sorted if requested, and only 15 papers starting from the given offset are converted to JSON.

```
    static boolean m(String alg, String t, String p) {
        switch (alg) { case "naive": return naive(t, p); case "rk": return rk(t, p); case "z":
            return z(t, p); default: return kmp(t, p); }
    }
```

```
    static String search(Map<String, String> q) throws IOException {
        final String qs = q.getOrDefault("q", "").toLowerCase().trim(), f =
q.getOrDefault("field",
            "all"), cat = q.getOrDefault("cat", ""), alg = q.getOrDefault("alg", "kmp");
        int off = Integer.parseInt(q.getOrDefault("offset", "0"));
        final boolean inT = f.equals("all") || f.equals("t"), inA = f.equals("all") ||
            f.equals("a"), inS = f.equals("all") || f.equals("s");
        String[] toks = qs.isEmpty() ? new String[0] : qs.split("\\s+");
        final String a2 = alg.equals("fuzzy") ? "kmp" : alg;
        final List<List<String>> alts = new ArrayList<>();
        for (String t : toks) alts.add(alg.equals("fuzzy") ? near(t) : List.of(t));
        if (off == 0 && !qs.isEmpty()) try (FileWriter fw = new FileWriter(HIST.toFile(), true))
{
            fw.write(qs + "\n"); }
        long t0 = System.nanoTime();
        List<Integer> hit = new ArrayList<>(IntStream.range(0, P.size()).parallel().filter(i -> {
            Paper p = P.get(i);
            if (!cat.isEmpty() && !p.c.equals(cat)) return false;
            for (List<String> al : alts) {
                boolean ok = false;
                for (String w : al) if ((inT && m(a2, p.lt, w)) || (inA && m(a2, p.la, w)) ||
(inS
                    && m(a2, p.ls, w))) { ok = true; break; }
                if (!ok) return false;
            }
            return true;
        }).boxed().collect(Collectors.toList()));
        double ms = (System.nanoTime() - t0) / 1e6;
        String sort = q.getOrDefault("sort", "r");
        if (sort.equals("n")) hit.sort((x, z) -> P.get(z).y - P.get(x).y);
        if (sort.equals("o")) hit.sort((x, z) -> P.get(x).y - P.get(z).y);
        String did = hit.isEmpty() && toks.length > 0 ?
            Arrays.stream(toks).map(ResearchConnect::dym).collect(Collectors.joining(" ")) : "";
        StringBuilder sb = new StringBuilder("{\"total\":" + hit.size() + ",\"ms\":" +
            String.format(Locale.ROOT, "%.1f", ms) + ",\"did\":" + j(did) + ",\"res\":[");
        for (int i = off; i < Math.min(hit.size(), off + 15); i++) { if (i > off) sb.append(',');
            sb.append(paperJson(P.get(hit.get(i)), -1)); }
        return sb.append("]}").toString();
    }
```

4.1.3 Fuzzy search (Edit Distance)

Edit Distance, also called Levenshtein distance, is the minimum number of insertions, deletions and substitutions needed to turn one word into another. For example "netwrk" and "network" have distance 1 because one letter must be inserted. It is calculated with dynamic programming. The full table has (n+1) x (m+1) cells, but each row depends only on the previous row, so the project keeps one row only and saves memory.

```
    static int ed(String a, String b) {
        int[] d = new int[b.length() + 1];
        for (int j = 0; j <= b.length(); j++) d[j] = j;
        for (int i = 1; i <= a.length(); i++) {
            int prev = d[0]; d[0] = i;
            for (int j = 1; j <= b.length(); j++) { int tmp = d[j]; d[j] = Math.min(Math.min(d[j]
+
                1, d[j - 1] + 1), prev + (a.charAt(i - 1) == b.charAt(j - 1) ? 0 : 1)); prev =
tmp;
                }
        }
        return d[b.length()];
    }
```

In fuzzy mode the method near() compares each query word with every word of the vocabulary of title words. A vocabulary word is accepted when its length differs from the query word by at most k and its edit distance is at most k, where k is 1 for words up to five letters and 2 for longer words. The five most frequent accepted words are used, and a paper matches if it contains the original word or any of these alternatives. For example, the query "neural netwrk" also finds papers that contain "neural network".

```
    static List<String> near(String t) {
        int k = t.length() > 5 ? 2 : 1; List<String> r = new ArrayList<>(List.of(t));
        r.addAll(V.entrySet().stream().filter(e -> !e.getKey().equals(t) &&
            Math.abs(e.getKey().length() - t.length()) <= k && ed(t, e.getKey()) <= k)
            .sorted((x, z) -> z.getValue() -
                x.getValue()).limit(5).map(Map.Entry::getKey).collect(Collectors.toList()));
        return r;
    }
```

When a normal search finds nothing, the method dym() ("did you mean") returns the vocabulary word with the smallest edit distance (two at most) to the query word; when several words are equally near, the more frequent one is chosen. The page then shows the suggestion as a link.

```
    static String dym(String t) {
        if (V.containsKey(t)) return t; String b = t; int bd = 3, bf = 0;
        for (Map.Entry<String, Integer> e : V.entrySet()) {
            String w = e.getKey();
            if (Math.abs(w.length() - t.length()) < 3) { int d = ed(t, w); if (d <= 2 && (d < bd
||
                (d == bd && e.getValue() > bf))) { bd = d; b = w; bf = e.getValue(); } }
        }
        return b;
    }
```

4.1.4 Search suggestions

After two typed letters the browser asks /api/suggest. The server returns up to five keywords that start with the last typed word (most frequent first), up to four paper titles that contain the typed text and up to three author names. When no keyword matches, the Edit Distance correction is offered instead. On the home page the suggestions appear in a dropdown under the search box; on the search page they fill a datalist.

```
    static String suggest(String raw) {
        String v = raw.toLowerCase().trim(); if (v.length() < 2) return "[]";
        String[] parts = v.split("\\s+"); String w = parts[parts.length - 1], pre =
v.substring(0,
            v.length() - w.length());
        List<String> o = new ArrayList<>();
        List<String> kw = V.entrySet().stream().filter(e -> e.getKey().startsWith(w)).sorted((x,
z)
            -> z.getValue() - x.getValue()).limit(5).map(e -> pre +
            e.getKey()).collect(Collectors.toList());
        for (String k : kw) o.add(sg(k, "keyword"));
        if (kw.isEmpty() && w.length() > 2) { String d = dym(w); if (!d.equals(w)) o.add(sg(pre +
d,
            "did you mean")); }
        int n = 0; for (Paper p : P) if (p.lt.contains(v)) { o.add(sg(p.t, "paper")); if (++n ==
4)
            break; }
        n = 0; for (String a : AU.keySet()) if (a.toLowerCase().contains(v)) { o.add(sg(a,
            "author")); if (++n == 3) break; }
        return arr(o.stream());
    }
```

4.1.5 Recommendation module

Recommendations are built from the papers the user saved or opened with the "Similar papers" button; these papers are called seeds. For every title word of a seed, the inverted index gives all papers that contain the word. Each shared word adds the score 1 / ln(2 + number of papers containing the word) to a paper, so rare, meaningful words such as "quantum" count more than common words such as "learning". A shared author adds 2 points and the same research domain adds 0.5 points. Words found in more than 5000 papers are skipped to keep the calculation fast. The papers with the highest scores, except the seeds, are returned together with a match percentage that is relative to the best result.

```
    static String recs(String ids, int k) {
        Set<Integer> seeds = new HashSet<>(); for (String s : ids.split(",")) if
(ID.containsKey(s))
            seeds.add(ID.get(s));
        if (seeds.isEmpty()) return "[]";
        Map<Integer, Double> sc = new HashMap<>(); Set<String> cats = new HashSet<>();
        for (int s : seeds) {
            Paper p = P.get(s); cats.add(p.c);
            for (String w : new HashSet<>(Arrays.asList(p.lt.split("[^a-z]+")))) { List<Integer>
l =
                IDX.get(w); if (l != null && l.size() < 5000) { double wt = 1 / Math.log(2 +
                l.size()); for (int x : l) sc.merge(x, wt, Double::sum); } }
            for (String a : p.a) { List<Integer> l = AU.get(a); if (l != null) for (int x : l)
                sc.merge(x, 2.0, Double::sum); }
        }
        sc.replaceAll((x, v) -> v + (cats.contains(P.get(x).c) ? .5 : 0));
        List<Map.Entry<Integer, Double>> top = sc.entrySet().stream().filter(e ->
            !seeds.contains(e.getKey())).sorted((x, z) -> Double.compare(z.getValue(),
            x.getValue())).limit(k).collect(Collectors.toList());
        if (top.isEmpty()) return "[]";
        double mx = top.get(0).getValue();
        return arr(top.stream().map(e -> paperJson(P.get(e.getKey()), Math.min(99, (int)
            (e.getValue() / mx * 100)))));
    }
```

4.1.6 Author network and influence

The authors form a graph in which every author is a node and two authors are connected when they wrote at least one paper together. The edges are not stored; when an author is opened, the collaborators are counted from the papers of that author. The influence score is the number of papers

plus the number of distinct collaborators (the degree of the node in the graph). The arXiv dataset has no citation links, so this score is based on collaboration and not on citations.

```
    static Map<String, Integer> collabs(String n) {
        Map<String, Integer> m = new HashMap<>();
        for (int i : AU.get(n)) for (String b : P.get(i).a) if (!b.equals(n) && !b.isEmpty())
            m.merge(b, 1, Integer::sum);
        return m;
    }
```

```
    static String author(String n) {
        List<Integer> l = AU.get(n); if (l == null) return "{}";
        Map<String, Integer> c = collabs(n);
        return "{\"n\":" + j(n) + ",\"c\":" + c.size() + ",\"papers\":" + arr(l.stream().map(i ->
            "{\"t\":" + j(P.get(i).t) + ",\"y\":" + P.get(i).y + "}"))
            + ",\"collabs\":" + arr(c.entrySet().stream().sorted((x, z) -> z.getValue() -
                x.getValue()).limit(15).map(e -> pair(e.getKey(), e.getValue()))) + "}";
    }
```

4.1.7 Analytics

The dashboard is produced by counting and sorting: papers per domain (from the TreeMap filled while loading), papers per year, the top authors by papers and collaborators, and the number of titles that contain each word (common stop words are ignored). The result is calculated once, kept in memory and reused for later requests.

```
    static synchronized String analytics() {
        if (analyticsCache == null) {
            String dom = arr(CAT.entrySet().stream().sorted((x, z) -> z.getValue() -
                x.getValue()).limit(12).map(e -> pair(e.getKey(), e.getValue())));
            Map<Integer, Integer> ym = new TreeMap<>(); for (Paper p : P) ym.merge(p.y, 1,
                Integer::sum);
            List<Map.Entry<Integer, Integer>> yl = new ArrayList<>(ym.entrySet());
            String yrs = arr(yl.subList(Math.max(0, yl.size() - 14), yl.size()).stream().map(e ->
                pair("" + e.getKey(), e.getValue())));
            String auth = arr(top().stream().limit(10).map(e -> pair(e.getKey(),
e.getValue().size()
                + collabs(e.getKey()).size())));
            String kws = arr(V.entrySet().stream().filter(e -> e.getKey().length() > 4 &&
                !STOP.contains(e.getKey())).sorted((x, z) -> z.getValue() -
                x.getValue()).limit(12).map(e -> pair(e.getKey(), e.getValue())));
            analyticsCache = "{\"d\":" + dom + ",\"y\":" + yrs + ",\"a\":" + auth + ",\"k\":" +
kws
                + "}";
        }
        return analyticsCache;
    }
```

4.1.8 Reading list and search history (file handling)

Pressing Save adds the paper id to saved.txt, and pressing the button again removes it. The file is read with Files.readAllLines, changed in memory and written back with Files.write. Every search is appended to history.txt with FileWriter in append mode, and the last six distinct searches are shown above the results.

```
    static synchronized String toggle(String id) throws IOException {
        List<String> l = lines(SAVED); boolean on = !l.remove(id); if (on) l.add(id);
        Files.write(SAVED, l, StandardCharsets.UTF_8); return "{\"saved\":" + on + "}";
    }
```

```
        if (off == 0 && !qs.isEmpty()) try (FileWriter fw = new FileWriter(HIST.toFile(), true))
{
            fw.write(qs + "\n"); }
```

4.1.9 HTTP server and API router

The main method loads the dataset, creates the HTTP server on port 8080 with a pool of four threads and registers one handler for the path "/". Paths that start with /api/ are passed to api(), which selects the feature; all other paths are answered with a file from the web folder. The check f.startsWith("web") prevents a request such as ../ from leaving the web folder.

```
    static String api(String name, Map<String, String> q) throws IOException {
        switch (name) {
            case "meta": return "{\"total\":" + P.size() + ",\"authors\":" + AU.size() +
                ",\"cats\":" + arr(CAT.keySet().stream().map(ResearchConnect::j)) + "}";
            case "search": return search(q);
            case "suggest": return suggest(q.getOrDefault("q", ""));
            case "recs": return recs(q.getOrDefault("ids", ""),
Integer.parseInt(q.getOrDefault("k",
                "8")));
            case "authors": return apiAuthors(q);
            case "author": return author(q.getOrDefault("name", ""));
            case "analytics": return analytics();
            case "save": return toggle(q.getOrDefault("id", ""));
            case "saved": return arr(lines(SAVED).stream().filter(ID::containsKey).map(i ->
                paperJson(P.get(ID.get(i)), -1)));
            case "history": { List<String> l = lines(HIST); Collections.reverse(l); return
                arr(l.stream().distinct().limit(6).map(ResearchConnect::j)); }
            default: return "{}";
        }
    }
```

```
    public static void main(String[] a) throws Exception {
        String file = a.length > 0 ? a[0] : "data/arXiv_scientific_dataset.csv"; int port =
a.length
            > 1 ? Integer.parseInt(a[1]) : 8080;
        Files.createDirectories(DIR);
        System.out.println("Loading " + file + " (about a minute for the full dataset)...");
        long t0 = System.currentTimeMillis(); load(file);
        System.out.println("Loaded " + P.size() + " papers in " + (System.currentTimeMillis() -
t0)
            / 1000 + "s");
        HttpServer s = HttpServer.create(new InetSocketAddress(port), 0);
        s.setExecutor(java.util.concurrent.Executors.newFixedThreadPool(4));
        s.createContext("/", ex -> {
            try {
                String path = ex.getRequestURI().getPath(); byte[] b; String type =
                    "application/json";
                if (path.startsWith("/api/")) b = api(path.substring(5),
                    query(ex.getRequestURI().getRawQuery())).getBytes(StandardCharsets.UTF_8);
                else {
                    Path f = Paths.get("web", path.equals("/") ? "index.html" :
                        path.substring(1)).normalize();
                    if (!f.startsWith("web") || !Files.isRegularFile(f)) {
                        ex.sendResponseHeaders(404, -1); return; }
                    b = Files.readAllBytes(f); type = path.endsWith(".js") ? "text/javascript" :
                        "text/html";
                }
                ex.getResponseHeaders().add("Content-Type", type + "; charset=utf-8");
                ex.sendResponseHeaders(200, b.length); ex.getResponseBody().write(b);
            } catch (Exception x) { x.printStackTrace(); try { ex.sendResponseHeaders(500, -1); }
                catch (Exception ignore) { } }
            finally { ex.close(); }
        });
        s.start();
        System.out.println("Open http://localhost:" + port);
    }
```

4.2 Front-end logic (UI rendering, state management)

The front end is plain JavaScript without any framework. It is made of the following parts.

- **Navigation:** the function go() shows one section, hides the others, highlights the active button and loads the data of the section when needed.
- **State:** a few variables keep the state: the set of saved paper ids, the set of papers opened with "Similar papers", the current query string and the number of results already shown.
- **Rendering:** the function card() builds the HTML of a paper card from a JSON object. All text is passed through an escape function before it is inserted, which prevents HTML injection.
- **Pagination:** the first request asks for offset 0, and the Show more button asks for the next 15 using the number of results already shown as the offset.
- **Event handling:** a single click listener on the document reads data attributes (data-sv, data-sim, data-au, data-h, data-sg), so buttons that are created later also work.
- **Suggestions:** typing calls /api/suggest and fills a dropdown (home page) or a datalist (search page).

```
function
    go(id,q){$
('#hs').style.display='none';document.querySelectorAll('section').forEach(s=>s.classList.toggle('
on',s.id===id));document.querySelectorAll('nav
    button').forEach(b=>b.classList.toggle('on',b.dataset.g===id));scrollTo(0,0);
if(q!==undefined){$
('#q').value=q;run()}if(id==='recs')recs();if(id==='sav')savedView();if(id==='net')authors();if(i
d==='ana'&&!$('#an').innerHTML)analytics()}
```

```
const card=(p,x='')=>`<div class="card p"><h3>${e(p.t)}</h3><div>${p.a.map(a=>`<a class="au"
data-
    au="${e(a)}">${e(a)}</a>`).join('')}</div>
```

```
async function run(){const q=$('#q').value.trim(),cat=$('#cat').value;
if(!q&&!cat){$('#info').innerHTML='<div class="note">Type a keyword or pick a domain to
    search.</div>';$('#res').innerHTML='';return}
qs=`q=${enc(q)}&field=${$('#fld').value}&cat=${enc(cat)}&alg=${$('#alg').value}&sort=${$
('#srt').value}`;shown=0;$('#res').innerHTML='';$('#info').innerHTML='<div
    class="mu">Searching…</div>';await more(true);hist()}
```

```
async function more(first){$('#more')?.remove();const r=await api(`search?${qs}&offset=$
{shown}`);
if(first){let i=`<div class="mu" style="margin-bottom:10px">${r.total.toLocaleString()} papers ·
    ${$('#alg').value.toUpperCase()} · ${r.ms} ms</div>`;
if(!r.total&&r.did)i+=`<div class="note">No matches. Did you mean <a class="au"
    data-h="${e(r.did)}"><b>${e(r.did)}</b></a>? You can also try Fuzzy
    search.</div>`;$('#info').innerHTML=i}
r.res.forEach(p=>$('#res').insertAdjacentHTML('beforeend',card(p)));shown+=r.res.length;
if(shown<r.total)$('#res').insertAdjacentHTML('beforeend','<button class="btn" id="more"
    onclick="more()">Show more</button>')}
```

4.3 API integration

The browser talks to the Java server with the fetch API. Every call returns JSON, which is turned

into HTML by the rendering functions. The helper below is used by all pages.

```
const api=u=>fetch('/api/'+u).then(r=>r.json());
```

For example, the search page sends the query, the selected field, domain, algorithm and sort order, together with the offset. The answer contains the total number of matches, the time spent in milliseconds, a "did you mean" word and the papers:

```
GET /api/search?q=neural%20network&field=all&cat=&alg=kmp&sort=r&offset=0

{ "total": 46941, "ms": 1361.6, "did": "",
  "res": [ {"id":"cs-9402101v1","t":"...","c":"Artificial Intelligence",
            "y":1994,"a":["C. X. Ling"],"s":"..."}, ... 15 papers ... ] }
```

The recommendation, author and analytics pages use the same pattern with their own endpoints from Table 3.2.

```
async function recs(){const ids=[...new Set([...saved,...viewed])].join(',');
$('#rl').innerHTML=ids?(await api('recs?ids='+enc(ids))).map(p=>card(p,`<span class="chip">match
    ${p.m}%</span>`)).join(''):'<div class="card mu">Nothing yet. Save papers or open "Similar
    papers" from the search page.</div>'}
```

```
async function author(n){go('net');const a=await api('author?name='+enc(n));if(!a.papers)return;
$('#ad').innerHTML=`<h3>${e(n)}</h3><div class="meta">${a.papers.length} papers · ${a.c}
    collaborators · influence score ${a.papers.length+a.c}</div><b>Collaborators</b><div
    style="margin:6px 0 12px">${a.collabs.map(c=>`<a class="au" data-au="${e(c[0])}">${e(c[0])}
    (${c[1]})</a>`).join('')||'<span class="mu">None in this
    dataset</span>'}</div><b>Papers</b>${a.papers.map(p=>`<div style="margin:6px 0">• ${e(p.t)}
    <span class="mu">(${p.y})</span></div>`).join('')}`}
```

4.4 Performance considerations

- The dataset is read once at start-up (about 34 seconds on the test computer) and then everything is searched in memory.
- The search uses parallel streams to use all CPU cores.
- Only 15 results are sent per request, so the response stays small.
- The analytics result and the list of top authors are calculated once and cached.
- Fuzzy search keeps only five alternatives per word to limit the amount of scanning.
- The inverted index skips words that occur in more than 5000 papers in the recommendation module.
- Only the first 400 characters of every abstract are kept for display, which saves memory (the full abstract is still searched).

4.5 Testing

The system was tested by calling every endpoint of the running Java program on the full dataset and by operating the web page in a browser. Table 4.2 lists the test cases and the observed results.

Table 4.2 Test cases and observed results

No.	Test	Observed result
T1	Open the home page	Statistics show 287,421 papers, 146 domains and 412,677 authors
T2	Search "neural network" with each of the four exact algorithms	All four return the same 46,941 papers
T3	Search "quantum", "reinforcement learning" and "graph" with each algorithm	Equal counts for all four algorithms (2,484; 19,105; 34,777)
T4	Fuzzy search "neural netwrk"	49,926 papers, which includes the papers for "neural network"
T5	Search "netwrk" with KMP	No match; the page offers "Did you mean network?"
T6	Search "robot" in the domain Robotics	3,781 papers
T7	Search "Hinton" in the field Author	32 papers
T8	Type "algor" on the search box	Keyword suggestions: algorithm, algorithms, algorithmic
T9	Save two papers and open Recommendations	Related papers are listed with a match percentage
T10	Open an author in Authors & Network	Papers, collaborators and influence score are shown
T11	Open the Reading List	Saved papers and the CSV text are shown; saved.txt is created
T12	Search a nonsense word	Zero matches

4.6 Running the project

1. Install JDK 17 or newer and the Extension Pack for Java in Visual Studio Code.
2. Copy arXiv_scientific_dataset.csv into the data folder of the project.
3. Run src/ResearchConnect.java in Visual Studio Code, or run run.bat (Windows) or run.sh (macOS and Linux).
4. Wait for the message "Open http://localhost:8080" and open that address in a browser.

4.7 Worked examples of the algorithms

This section shows the algorithms on small inputs, calculated by the same logic as in the program, so that their working can be followed step by step.

4.7.1 KMP failure table

For the pattern "abcabd" the failure table is built as follows. The value at position i is the length of the longest proper prefix of the pattern that is also a suffix of its first i+1 letters.

Table 4.3 Failure table of the pattern "abcabd"

Position i	0	1	2	3	4	5
Letter	a	b	c	a	b	d
f[i]	0	0	0	1	2	0

When this pattern is searched in the text "abcabcabd", the first mismatch happens at the sixth letter (d against c) after five letters matched. Instead of starting again at the second text letter, KMP uses f[4] = 2 and continues with the pattern letter at position 2, without reading the text again. After the shift the remaining letters match and the pattern is found at position 3 of the text.

4.7.2 Edit Distance table

The full table for turning "netwrk" into "network" is shown in Table 4.3. The program keeps only one row at a time, but the full table makes the idea clear: every cell is the smallest of the cell above plus one (deletion), the cell to the left plus one (insertion) and the diagonal cell plus zero or one (match or substitution). The value in the last cell is the edit distance, 1.

Table 4.4 Edit Distance table for "netwrk" and "network"

		n	e	t	w	o	r	k
	0	1	2	3	4	5	6	7
n	1	0	1	2	3	4	5	6
e	2	1	0	1	2	3	4	5
t	3	2	1	0	1	2	3	4
w	4	3	2	1	0	1	2	3
r	5	4	3	2	1	1	1	2
k	6	5	4	3	2	2	2	1

Because the distance is 1 and the word has more than five letters (limit k = 2), "netwrk" would be accepted as a near spelling of "network" by the fuzzy search, and the program also finds the correct word as a "Did you mean" suggestion.

4.7.3 Z array

To search the pattern "aab" in the text "baabaa", the program builds the string "aab#baabaa" and calculates its Z array (Table 4.4). A value equal to the length of the pattern (3) shows a match; here it appears at position 5 of the combined string, which is position 1 of the text.

Table 4.5 Z array of "aab#baabaa"

Position	0	1	2	3	4	5	6	7	8	9
Letter	a	a	b	#	b	a	a	b	a	a
Z[i]	0	1	0	0	0	3	1	0	2	1

4.7.4 Rabin-Karp rolling hash

With base 256 and modulus 1000003 the hash of the pattern "bc" is 25187. Table 4.5 shows the hashes of the windows of the text "abcd". Only the window whose hash equals the pattern hash is compared letter by letter. In the program the next hash is not recalculated from the beginning but is obtained from the previous one by removing the first letter and adding the next letter.

Table 4.6 Window hashes in Rabin-Karp

Start	Window	Hash	Equal to pattern hash?
0	ab	24930	no
1	bc	25187	yes
2	cd	25444	no

4.7.5 Recommendation score

Suppose a saved paper has the title word "quantum" and that word occurs in 300 titles of the dataset, while the word "learning" occurs in 51,652 titles. The weight 1 / ln(2 + postings) is 0.175 for "quantum" and 0.092 for "learning". A paper that shares the word "quantum" with the saved paper therefore gets about 1.9 times the score of a paper that shares only "learning". If the two papers also have a common author the score rises by 2, and by 0.5 if the domain is the same. This is why the recommendations favour specific topics.

4.8 Additional front-end code

The remaining front-end functions are shown here for completeness. The function sug() fills the suggestion dropdown, authors() loads the author list, the function bars() draws the analytics charts with plain HTML, and the click listener handles all buttons.

```
async function sug(inp,box,dl){const
    v=inp.value.trim();if(v.length<2){if(box)box.style.display='none';return}const l=await
    api('suggest?q='+enc(v));
if(dl)dl.innerHTML=l.filter(x=>x.k==='keyword').map(x=>`<option value="${e(x.v)}">`).join('');
if(box){box.innerHTML=l.map(x=>`<div data-

sg="${e(x.v)}"><span>${e(x.v)}</span><em>${x.k}</em></div>`).join('');box.style.display=l.length?
'block':'none'}}
```

```
async function authors(){const l=await api('authors?q='+enc($('#aq').value));
$('#al').innerHTML=l.map(a=>`<div class="card" style="margin-bottom:8px;cursor:pointer" data-
    au="${e(a.n)}"><b>${e(a.n)}</b><div class="meta">${a.p} papers · ${a.c} collaborators ·
    influence ${a.p+a.c}</div></div>`).join('')}
```

```
const bars=(t,en)=>{const mx=Math.max(...en.map(x=>x[1]));return `<div
    class="card"><h3>${t}</h3>${en.map(x=>`<div class="row"><span
    title="${e(x[0])}">${e(x[0])}</span><i
    style="width:${x[1]/mx*100}%"></i><span>${x[1]}</span></div>`).join('')}</div>`};

async function analytics(){const a=await api('analytics');$('#an').innerHTML=bars('Papers by
    domain',a.d)+bars('Top authors (influence)',a.a)+bars('Papers per year',a.y)+bars('Trending
    keywords',a.k)}

async function savedView(){const l=await
    api('saved');$('#sl').innerHTML=l.map(p=>card(p)).join('')||'<div class="card mu">No saved
    papers yet.</div>';
$('#csv').value='id,title,category,year\n'+l.map(p=>[p.id,'"'+p.t.replace(/"/g,'""')
+'"',p.c,p.y].join(',')).join('\n')}
```

4.9 Complete backend source code

The complete source code of the backend, src/ResearchConnect.java, is given below so that the report is self-contained. Long lines are wrapped for printing.

```
import com.sun.net.httpserver.HttpServer;
import java.io.*;
import java.net.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.*;
import java.util.stream.*;
/** Research Connect backend. NO database: the arXiv CSV is streamed with BufferedReader,
 *  saved papers (data/saved.txt) and search history (data/history.txt) are plain text files. */
public class ResearchConnect {
    static class Paper { String id, t, c, s, lt, la, ls; int y; String[] a; }
    static final List<Paper> P = new ArrayList<>();
    static final Map<String, Integer> ID = new HashMap<>(), V = new HashMap<>(), CAT = new
        TreeMap<>();
    static final Map<String, List<Integer>> AU = new HashMap<>(), IDX = new HashMap<>();
    static final Set<String> STOP = new HashSet<>(Arrays.asList(("the and for that with this are
        from which have been their can our using use based into also these such than more not but
        was were its has new one two may how between over only each other both under via show shown
        paper present approach results method study problem").split(" ")));
    static final Path DIR = Paths.get("data"), SAVED = DIR.resolve("saved.txt"), HIST =
        DIR.resolve("history.txt");
    static String analyticsCache;
    static List<Map.Entry<String, List<Integer>>> topList;
    // ---------- file handling: stream the CSV (quoted fields may span several lines) ----------
    static void load(String file) throws IOException {
        try (BufferedReader br = new BufferedReader(new FileReader(file, StandardCharsets.UTF_8), 1
            << 16)) {
            br.readLine();
            StringBuilder cur = new StringBuilder(); List<String> f = new ArrayList<>(); boolean q =
                false; int c;
            while ((c = br.read()) != -1) {
                if (c == '"') q = !q;
                else if (c == ',' && !q) { f.add(cur.toString()); cur.setLength(0); }
                else if (c == '\n' && !q) { f.add(cur.toString()); cur.setLength(0); add(f);
                    f.clear(); }
                else if (c != '\r') cur.append((char) c);
            }
        }
    }
    static String[] parseAuthors(String s) {
        s = s.trim();
        if (s.startsWith("[") && s.endsWith("]")) s = s.substring(1, s.length() - 1);
        String[] r = s.split("['\"],\\s*['\"]");
        for (int i = 0; i < r.length; i++) r[i] = r[i].replaceAll("^['\"]|['\"]$", "").trim();
        return r;
    }
    static void add(List<String> f) {
        if (f.size() < 9) return;
        Paper p = new Paper(); p.id = f.get(0); p.t = f.get(1).replaceAll("\\s+", " ").trim(); p.c =
            f.get(2);
        String[] d = f.get(4).split("/"); int y;
        try { y = Integer.parseInt(d[d.length - 1].trim()); } catch (Exception ex) { return; }
        p.y = y < 100 ? (y > 26 ? 1900 + y : 2000 + y) : y;
        p.a = parseAuthors(f.get(6));
        String s = f.get(8).replaceAll("\\s+", " ").trim();
        p.ls = s.toLowerCase(); p.s = s.length() > 400 ? s.substring(0, 400) : s;
        p.lt = p.t.toLowerCase(); p.la = String.join(" ", p.a).toLowerCase();
```

```
        int i = P.size(); P.add(p); ID.put(p.id, i); CAT.merge(p.c, 1, Integer::sum);
        for (String a : new LinkedHashSet<>(Arrays.asList(p.a))) if (!a.isEmpty())
            AU.computeIfAbsent(a, k -> new ArrayList<>()).add(i);
        for (String w : new HashSet<>(Arrays.asList(p.lt.split("[^a-z]+"))))
            if (w.length() >= 3) { V.merge(w, 1, Integer::sum); if (w.length() >= 4 &&
                !STOP.contains(w)) IDX.computeIfAbsent(w, k -> new ArrayList<>()).add(i); }
    }
    static List<String> lines(Path p) throws IOException { return Files.exists(p) ?
        Files.readAllLines(p, StandardCharsets.UTF_8) : new ArrayList<>(); }
    // ---------- pattern matching + edit distance ----------
    static boolean naive(String t, String p) {
        for (int i = 0; i <= t.length() - p.length(); i++) { int j = 0; while (j < p.length() &&
            t.charAt(i + j) == p.charAt(j)) j++; if (j == p.length()) return true; }
        return false;
    }
    static boolean kmp(String t, String p) {
        int[] f = new int[p.length()];
        for (int i = 1, k = 0; i < p.length(); i++) { while (k > 0 && p.charAt(i) != p.charAt(k)) k
            = f[k - 1]; if (p.charAt(i) == p.charAt(k)) k++; f[i] = k; }
        for (int i = 0, k = 0; i < t.length(); i++) { while (k > 0 && t.charAt(i) != p.charAt(k)) k
            = f[k - 1]; if (t.charAt(i) == p.charAt(k)) k++; if (k == p.length()) return true; }
        return false;
    }
    static boolean rk(String t, String p) {
        int m = p.length(), n = t.length(); if (m > n) return false;
        final long M = 1000003; long h = 0, w = 0, pw = 1;
        for (int i = 0; i < m; i++) { h = (h * 256 + p.charAt(i)) % M; w = (w * 256 + t.charAt(i)) %
            M; if (i > 0) pw = pw * 256 % M; }
        for (int i = 0; ; i++) {
            if (h == w && t.startsWith(p, i)) return true;
            if (i + m >= n) return false;
            w = ((w - t.charAt(i) * pw % M + M) * 256 + t.charAt(i + m)) % M;
        }
    }
    static boolean z(String t, String p) {
        String s = p + "\u0001" + t; int n = s.length(), m = p.length(); int[] z = new int[n];
        for (int i = 1, l = 0, r = 0; i < n; i++) {
            if (i < r) z[i] = Math.min(r - i, z[i - l]);
            while (i + z[i] < n && s.charAt(z[i]) == s.charAt(i + z[i])) z[i]++;
            if (i + z[i] > r) { l = i; r = i + z[i]; }
            if (z[i] == m) return true;
        }
        return false;
    }
    static boolean m(String alg, String t, String p) {
        switch (alg) { case "naive": return naive(t, p); case "rk": return rk(t, p); case "z":
            return z(t, p); default: return kmp(t, p); }
    }
    static int ed(String a, String b) {
        int[] d = new int[b.length() + 1];
        for (int j = 0; j <= b.length(); j++) d[j] = j;
        for (int i = 1; i <= a.length(); i++) {
            int prev = d[0]; d[0] = i;
            for (int j = 1; j <= b.length(); j++) { int tmp = d[j]; d[j] = Math.min(Math.min(d[j] +
                1, d[j - 1] + 1), prev + (a.charAt(i - 1) == b.charAt(j - 1) ? 0 : 1)); prev = tmp;
                }
        }
        return d[b.length()];
    }
    static List<String> near(String t) {
        int k = t.length() > 5 ? 2 : 1; List<String> r = new ArrayList<>(List.of(t));
        r.addAll(V.entrySet().stream().filter(e -> !e.getKey().equals(t) &&
            Math.abs(e.getKey().length() - t.length()) <= k && ed(t, e.getKey()) <= k)
            .sorted((x, z) -> z.getValue() -
                x.getValue()).limit(5).map(Map.Entry::getKey).collect(Collectors.toList()));
        return r;
    }
    static String dym(String t) {
        if (V.containsKey(t)) return t; String b = t; int bd = 3, bf = 0;
        for (Map.Entry<String, Integer> e : V.entrySet()) {
            String w = e.getKey();
            if (Math.abs(w.length() - t.length()) < 3) { int d = ed(t, w); if (d <= 2 && (d < bd ||
                (d == bd && e.getValue() > bf))) { bd = d; b = w; bf = e.getValue(); } }
        }
        return b;
    }
    // ---------- JSON helpers ----------
    static String j(String s) {
        StringBuilder b = new StringBuilder("\"");
        for (char c : s.toCharArray()) { if (c == '"' || c == '\\') b.append('\\').append(c); else
            if (c < 32) b.append(' '); else b.append(c); }
        return b.append('"').toString();
    }
    static String arr(Stream<String> s) { return s.collect(Collectors.joining(",", "[", "]")); }
    static String pair(String n, int v) { return "[" + j(n) + "," + v + "]"; }
    static String paperJson(Paper p, int m) {
        return "{\"id\":" + j(p.id) + ",\"t\":" + j(p.t) + ",\"c\":" + j(p.c) + ",\"y\":" + p.y +
            ",\"a\":" + arr(Arrays.stream(p.a).limit(8).map(ResearchConnect::j))
            + ",\"s\":" + j(p.s) + (m >= 0 ? ",\"m\":" + m : "") + "}";
```

```
    }
    // ---------- features ----------
    static String search(Map<String, String> q) throws IOException {
        final String qs = q.getOrDefault("q", "").toLowerCase().trim(), f = q.getOrDefault("field",
            "all"), cat = q.getOrDefault("cat", ""), alg = q.getOrDefault("alg", "kmp");
        int off = Integer.parseInt(q.getOrDefault("offset", "0"));
        final boolean inT = f.equals("all") || f.equals("t"), inA = f.equals("all") ||
            f.equals("a"), inS = f.equals("all") || f.equals("s");
        String[] toks = qs.isEmpty() ? new String[0] : qs.split("\\s+");
        final String a2 = alg.equals("fuzzy") ? "kmp" : alg;
        final List<List<String>> alts = new ArrayList<>();
        for (String t : toks) alts.add(alg.equals("fuzzy") ? near(t) : List.of(t));
        if (off == 0 && !qs.isEmpty()) try (FileWriter fw = new FileWriter(HIST.toFile(), true)) {
            fw.write(qs + "\n"); }
        long t0 = System.nanoTime();
        List<Integer> hit = new ArrayList<>(IntStream.range(0, P.size()).parallel().filter(i -> {
            Paper p = P.get(i);
            if (!cat.isEmpty() && !p.c.equals(cat)) return false;
            for (List<String> al : alts) {
                boolean ok = false;
                for (String w : al) if ((inT && m(a2, p.lt, w)) || (inA && m(a2, p.la, w)) || (inS
                    && m(a2, p.ls, w))) { ok = true; break; }
                if (!ok) return false;
            }
            return true;
        }).boxed().collect(Collectors.toList()));
        double ms = (System.nanoTime() - t0) / 1e6;
        String sort = q.getOrDefault("sort", "r");
        if (sort.equals("n")) hit.sort((x, z) -> P.get(z).y - P.get(x).y);
        if (sort.equals("o")) hit.sort((x, z) -> P.get(x).y - P.get(z).y);
        String did = hit.isEmpty() && toks.length > 0 ?
            Arrays.stream(toks).map(ResearchConnect::dym).collect(Collectors.joining(" ")) : "";
        StringBuilder sb = new StringBuilder("{\"total\":" + hit.size() + ",\"ms\":" +
            String.format(Locale.ROOT, "%.1f", ms) + ",\"did\":" + j(did) + ",\"res\":[");
        for (int i = off; i < Math.min(hit.size(), off + 15); i++) { if (i > off) sb.append(',');
            sb.append(paperJson(P.get(hit.get(i)), -1)); }
        return sb.append("]}").toString();
    }
    static String sg(String v, String k) { return "{\"v\":" + j(v) + ",\"k\":" + j(k) + "}"; }
    static String suggest(String raw) {
        String v = raw.toLowerCase().trim(); if (v.length() < 2) return "[]";
        String[] parts = v.split("\\s+"); String w = parts[parts.length - 1], pre = v.substring(0,
            v.length() - w.length());
        List<String> o = new ArrayList<>();
        List<String> kw = V.entrySet().stream().filter(e -> e.getKey().startsWith(w)).sorted((x, z)
            -> z.getValue() - x.getValue()).limit(5).map(e -> pre +
            e.getKey()).collect(Collectors.toList());
        for (String k : kw) o.add(sg(k, "keyword"));
        if (kw.isEmpty() && w.length() > 2) { String d = dym(w); if (!d.equals(w)) o.add(sg(pre + d,
            "did you mean")); }
        int n = 0; for (Paper p : P) if (p.lt.contains(v)) { o.add(sg(p.t, "paper")); if (++n == 4)
            break; }
        n = 0; for (String a : AU.keySet()) if (a.toLowerCase().contains(v)) { o.add(sg(a,
            "author")); if (++n == 3) break; }
        return arr(o.stream());
    }
    /** Recommendations: inverted index on title words (idf-weighted) + shared authors + same
        domain. */
    static String recs(String ids, int k) {
        Set<Integer> seeds = new HashSet<>(); for (String s : ids.split(",")) if (ID.containsKey(s))
            seeds.add(ID.get(s));
        if (seeds.isEmpty()) return "[]";
        Map<Integer, Double> sc = new HashMap<>(); Set<String> cats = new HashSet<>();
        for (int s : seeds) {
            Paper p = P.get(s); cats.add(p.c);
            for (String w : new HashSet<>(Arrays.asList(p.lt.split("[^a-z]+")))) { List<Integer> l =
                IDX.get(w); if (l != null && l.size() < 5000) { double wt = 1 / Math.log(2 +
                l.size()); for (int x : l) sc.merge(x, wt, Double::sum); } }
            for (String a : p.a) { List<Integer> l = AU.get(a); if (l != null) for (int x : l)
                sc.merge(x, 2.0, Double::sum); }
        }
        sc.replaceAll((x, v) -> v + (cats.contains(P.get(x).c) ? .5 : 0));
        List<Map.Entry<Integer, Double>> top = sc.entrySet().stream().filter(e ->
            !seeds.contains(e.getKey())).sorted((x, z) -> Double.compare(z.getValue(),
            x.getValue())).limit(k).collect(Collectors.toList());
        if (top.isEmpty()) return "[]";
        double mx = top.get(0).getValue();
        return arr(top.stream().map(e -> paperJson(P.get(e.getKey()), Math.min(99, (int)
            (e.getValue() / mx * 100)))));
    }
    static Map<String, Integer> collabs(String n) {
        Map<String, Integer> m = new HashMap<>();
        for (int i : AU.get(n)) for (String b : P.get(i).a) if (!b.equals(n) && !b.isEmpty())
            m.merge(b, 1, Integer::sum);
        return m;
    }
    static synchronized List<Map.Entry<String, List<Integer>>> top() {
        if (topList == null) topList = AU.entrySet().stream().sorted((x, z) -> z.getValue().size() -
            x.getValue().size()).limit(25).collect(Collectors.toList());
```

```
        return topList;
    }
    static String apiAuthors(Map<String, String> q) {
        String f = q.getOrDefault("q", "").toLowerCase().trim();
        List<Map.Entry<String, List<Integer>>> l = f.isEmpty() ? top() :
            AU.entrySet().stream().filter(e -> e.getKey().toLowerCase().contains(f))
            .sorted((x, z) -> z.getValue().size() -
                x.getValue().size()).limit(25).collect(Collectors.toList());
        return arr(l.stream().map(e -> "{\"n\":" + j(e.getKey()) + ",\"p\":" + e.getValue().size() +
            ",\"c\":" + collabs(e.getKey()).size() + "}"));
    }
    static String author(String n) {
        List<Integer> l = AU.get(n); if (l == null) return "{}";
        Map<String, Integer> c = collabs(n);
        return "{\"n\":" + j(n) + ",\"c\":" + c.size() + ",\"papers\":" + arr(l.stream().map(i ->
            "{\"t\":" + j(P.get(i).t) + ",\"y\":" + P.get(i).y + "}"))
            + ",\"collabs\":" + arr(c.entrySet().stream().sorted((x, z) -> z.getValue() -
                x.getValue()).limit(15).map(e -> pair(e.getKey(), e.getValue()))) + "}";
    }
    static synchronized String analytics() {
        if (analyticsCache == null) {
            String dom = arr(CAT.entrySet().stream().sorted((x, z) -> z.getValue() -
                x.getValue()).limit(12).map(e -> pair(e.getKey(), e.getValue())));
            Map<Integer, Integer> ym = new TreeMap<>(); for (Paper p : P) ym.merge(p.y, 1,
                Integer::sum);
            List<Map.Entry<Integer, Integer>> yl = new ArrayList<>(ym.entrySet());
            String yrs = arr(yl.subList(Math.max(0, yl.size() - 14), yl.size()).stream().map(e ->
                pair("" + e.getKey(), e.getValue())));
            String auth = arr(top().stream().limit(10).map(e -> pair(e.getKey(), e.getValue().size()
                + collabs(e.getKey()).size())));
            String kws = arr(V.entrySet().stream().filter(e -> e.getKey().length() > 4 &&
                !STOP.contains(e.getKey())).sorted((x, z) -> z.getValue() -
                x.getValue()).limit(12).map(e -> pair(e.getKey(), e.getValue())));
            analyticsCache = "{\"d\":" + dom + ",\"y\":" + yrs + ",\"a\":" + auth + ",\"k\":" + kws
                + "}";
        }
        return analyticsCache;
    }
    static synchronized String toggle(String id) throws IOException {
        List<String> l = lines(SAVED); boolean on = !l.remove(id); if (on) l.add(id);
        Files.write(SAVED, l, StandardCharsets.UTF_8); return "{\"saved\":" + on + "}";
    }
    static String api(String name, Map<String, String> q) throws IOException {
        switch (name) {
            case "meta": return "{\"total\":" + P.size() + ",\"authors\":" + AU.size() +
                ",\"cats\":" + arr(CAT.keySet().stream().map(ResearchConnect::j)) + "}";
            case "search": return search(q);
            case "suggest": return suggest(q.getOrDefault("q", ""));
            case "recs": return recs(q.getOrDefault("ids", ""), Integer.parseInt(q.getOrDefault("k",
                "8")));
            case "authors": return apiAuthors(q);
            case "author": return author(q.getOrDefault("name", ""));
            case "analytics": return analytics();
            case "save": return toggle(q.getOrDefault("id", ""));
            case "saved": return arr(lines(SAVED).stream().filter(ID::containsKey).map(i ->
                paperJson(P.get(ID.get(i)), -1)));
            case "history": { List<String> l = lines(HIST); Collections.reverse(l); return
                arr(l.stream().distinct().limit(6).map(ResearchConnect::j)); }
            default: return "{}";
        }
    }
    static Map<String, String> query(String raw) {
        Map<String, String> m = new HashMap<>(); if (raw == null) return m;
        for (String kv : raw.split("&")) { String[] p = kv.split("=", 2); if (p.length == 2)
            m.put(p[0], URLDecoder.decode(p[1], StandardCharsets.UTF_8)); }
        return m;
    }
    public static void main(String[] a) throws Exception {
        String file = a.length > 0 ? a[0] : "data/arXiv_scientific_dataset.csv"; int port = a.length
            > 1 ? Integer.parseInt(a[1]) : 8080;
        Files.createDirectories(DIR);
        System.out.println("Loading " + file + " (about a minute for the full dataset)...");
        long t0 = System.currentTimeMillis(); load(file);
        System.out.println("Loaded " + P.size() + " papers in " + (System.currentTimeMillis() - t0)
            / 1000 + "s");
        HttpServer s = HttpServer.create(new InetSocketAddress(port), 0);
        s.setExecutor(java.util.concurrent.Executors.newFixedThreadPool(4));
        s.createContext("/", ex -> {
            try {
                String path = ex.getRequestURI().getPath(); byte[] b; String type =
                    "application/json";
                if (path.startsWith("/api/")) b = api(path.substring(5),
                    query(ex.getRequestURI().getRawQuery())).getBytes(StandardCharsets.UTF_8);
                else {
                    Path f = Paths.get("web", path.equals("/") ? "index.html" :
                        path.substring(1)).normalize();
                    if (!f.startsWith("web") || !Files.isRegularFile(f)) {
                        ex.sendResponseHeaders(404, -1); return; }
                    b = Files.readAllBytes(f); type = path.endsWith(".js") ? "text/javascript" :
```

```
                        "text/html";
                }
                ex.getResponseHeaders().add("Content-Type", type + "; charset=utf-8");
                ex.sendResponseHeaders(200, b.length); ex.getResponseBody().write(b);
            } catch (Exception x) { x.printStackTrace(); try { ex.sendResponseHeaders(500, -1); }
                catch (Exception ignore) { } }
            finally { ex.close(); }
        });
        s.start();
        System.out.println("Open http://localhost:" + port);
    }
}
```

4.10 Front-end source code

The JavaScript file web/app.js, which draws the pages and talks to the Java server, is given below. The file index.html contains the page structure and the style sheet.

```
const

$=s=>document.querySelector(s),e=s=>String(s).replace(/[&<>"]/g,c=>({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&q
uot;'}[c])),enc=encodeURIComponent;
const api=u=>fetch('/api/'+u).then(r=>r.json());
let saved=new Set(),viewed=new Set(),shown=0,qs='';
const NAV=[['home','Home'],['search','Search'],['recs','Recommendations'],['net','Authors &
    Network'],['ana','Analytics'],['sav','Reading List']];
$('#nav').innerHTML=NAV.map(n=>`<button data-g="${n[0]}">${n[1]}</button>`).join('');
$('#nav').onclick=ev=>{if(ev.target.dataset.g)go(ev.target.dataset.g)};
function
    go(id,q){$
('#hs').style.display='none';document.querySelectorAll('section').forEach(s=>s.classList.toggle('on',s.id===id
));document.querySelectorAll('nav
    button').forEach(b=>b.classList.toggle('on',b.dataset.g===id));scrollTo(0,0);
if(q!==undefined){$
('#q').value=q;run()}if(id==='recs')recs();if(id==='sav')savedView();if(id==='net')authors();if(id==='ana'&&!$
('#an').innerHTML)analytics()}
const card=(p,x='')=>`<div class="card p"><h3>${e(p.t)}</h3><div>${p.a.map(a=>`<a class="au" data-
    au="${e(a)}">${e(a)}</a>`).join('')}</div>
<div class="meta"><span class="chip">${e(p.c)}</span><span class="chip">${p.y}</span><span
    class="chip">${e(p.id)}</span>${x}</div><div class="mu">${e(p.s)}${p.s.length>=400?'…':''}</div>
<div style="margin-top:8px"><button class="btn s" data-sv="${e(p.id)}">${saved.has(p.id)?'★
    Saved':'☆ Save'}</button> <button class="btn s" data-sim="${e(p.id)}">Similar
    papers</button></div></div>`;
/* search (all work is done by Java; the page only asks for 15 results at a time) */
async function run(){const q=$('#q').value.trim(),cat=$('#cat').value;
if(!q&&!cat){$('#info').innerHTML='<div class="note">Type a keyword or pick a domain to
    search.</div>';$('#res').innerHTML='';return}
qs=`q=${enc(q)}&field=${$('#fld').value}&cat=${enc(cat)}&alg=${$('#alg').value}&sort=${$
('#srt').value}`;shown=0;$('#res').innerHTML='';$('#info').innerHTML='<div
    class="mu">Searching…</div>';await more(true);hist()}
async function more(first){$('#more')?.remove();const r=await api(`search?${qs}&offset=${shown}`);
if(first){let i=`<div class="mu" style="margin-bottom:10px">${r.total.toLocaleString()} papers ·
    ${$('#alg').value.toUpperCase()} · ${r.ms} ms</div>`;
if(!r.total&&r.did)i+=`<div class="note">No matches. Did you mean <a class="au"
    data-h="${e(r.did)}"><b>${e(r.did)}</b></a>? You can also try Fuzzy
    search.</div>`;$('#info').innerHTML=i}
r.res.forEach(p=>$('#res').insertAdjacentHTML('beforeend',card(p)));shown+=r.res.length;
if(shown<r.total)$('#res').insertAdjacentHTML('beforeend','<button class="btn" id="more"
    onclick="more()">Show more</button>')}
const hist=()=>api('history').then(h=>$('#hist').innerHTML=h.length?'Recent: '+h.map(x=>`<a
    class="au" data-h="${e(x)}">${e(x)}</a>`).join(''):'');
$('#go').onclick=run;$('#q').onkeydown=ev=>{if(ev.key==='Enter')run()};$
('#hq').onkeydown=ev=>{if(ev.key==='Enter')go('search',$('#hq').value)};
/* suggestions: dropdown under the home box, datalist on the search page */
async function sug(inp,box,dl){const
    v=inp.value.trim();if(v.length<2){if(box)box.style.display='none';return}const l=await
    api('suggest?q='+enc(v));
if(dl)dl.innerHTML=l.filter(x=>x.k==='keyword').map(x=>`<option value="${e(x.v)}">`).join('');
if(box){box.innerHTML=l.map(x=>`<div data-

sg="${e(x.v)}"><span>${e(x.v)}</span><em>${x.k}</em></div>`).join('');box.style.display=l.length?'block':'none
'}}
$('#hq').oninput=()=>sug($('#hq'),$('#hs'));$('#q').oninput=()=>sug($('#q'),null,$('#sg'));
/* recommendations */
async function recs(){const ids=[...new Set([...saved,...viewed])].join(',');
$('#rl').innerHTML=ids?(await api('recs?ids='+enc(ids))).map(p=>card(p,`<span class="chip">match
    ${p.m}%</span>`)).join(''):'<div class="card mu">Nothing yet. Save papers or open "Similar
    papers" from the search page.</div>'}
/* authors & network */
async function authors(){const l=await api('authors?q='+enc($('#aq').value));
$('#al').innerHTML=l.map(a=>`<div class="card" style="margin-bottom:8px;cursor:pointer" data-
    au="${e(a.n)}"><b>${e(a.n)}</b><div class="meta">${a.p} papers · ${a.c} collaborators ·
    influence ${a.p+a.c}</div></div>`).join('')}
```

```
$('#aq').oninput=authors;
async function author(n){go('net');const a=await api('author?name='+enc(n));if(!a.papers)return;
$('#ad').innerHTML=`<h3>${e(n)}</h3><div class="meta">${a.papers.length} papers · ${a.c}
    collaborators · influence score ${a.papers.length+a.c}</div><b>Collaborators</b><div
    style="margin:6px 0 12px">${a.collabs.map(c=>`<a class="au" data-au="${e(c[0])}">${e(c[0])}
    (${c[1]})</a>`).join('')||'<span class="mu">None in this
    dataset</span>'}</div><b>Papers</b>${a.papers.map(p=>`<div style="margin:6px 0">• ${e(p.t)}
    <span class="mu">(${p.y})</span></div>`).join('')}`}
/* analytics */
const bars=(t,en)=>{const mx=Math.max(...en.map(x=>x[1]));return `<div
    class="card"><h3>${t}</h3>${en.map(x=>`<div class="row"><span
    title="${e(x[0])}">${e(x[0])}</span><i
    style="width:${x[1]/mx*100}%"></i><span>${x[1]}</span></div>`).join('')}</div>`};
async function analytics(){const a=await api('analytics');$('#an').innerHTML=bars('Papers by
    domain',a.d)+bars('Top authors (influence)',a.a)+bars('Papers per year',a.y)+bars('Trending
    keywords',a.k)}
/* reading list (stored by Java in data/saved.txt) */
async function savedView(){const l=await
    api('saved');$('#sl').innerHTML=l.map(p=>card(p)).join('')||'<div class="card mu">No saved
    papers yet.</div>';
$('#csv').value='id,title,category,year\n'+l.map(p=>[p.id,'"'+p.t.replace(/"/g,'""')
+'"',p.c,p.y].join(',')).join('\n')}
document.addEventListener('click',async ev=>{const t=ev.target.closest('[data-sg],[data-
    au],[data-h],[data-sv],[data-sim]')||ev.target,d=t.dataset;
if(!ev.target.closest('#hs')&&ev.target.id!=='hq')$('#hs').style.display='none';
if(d.sg!==undefined)go('search',d.sg);if(d.au)author(d.au);if(d.h!==undefined)go('search',d.h);
if(d.sv!==undefined){const r=await
    api('save?id='+enc(d.sv));r.saved?saved.add(d.sv):saved.delete(d.sv);t.textContent=r.saved?'★
    Saved':'☆ Save';if($('#sav').classList.contains('on'))savedView()}
if(d.sim!==undefined){viewed.add(d.sim);const
    c=t.closest('.card');c.nextElementSibling?.classList.contains('simx')&&c.nextElementSibling.remove();
const l=await api('recs?k=4&ids='+enc(d.sim));c.insertAdjacentHTML('afterend',`<div class="simx"
    style="margin-left:22px">${l.map(p=>card(p,`<span class="chip">match
    ${p.m}%</span>`)).join('')}</div>`)}});
/* start-up */
api('meta').then(m=>{m.cats.forEach(c=>$('#cat').insertAdjacentHTML('beforeend',`<option>${e(c)}</option>`));
$('#stats').innerHTML=[[m.total.toLocaleString(),'papers in the dataset'],[m.cats.length,'research
    domains'],[m.authors.toLocaleString(),'authors in network']].map(s=>`<div class="card
    stat"><b>${s[0]}</b>${s[1]}</div>`).join('')});
api('saved').then(l=>l.forEach(p=>saved.add(p.id)));hist();
```

5. RESULTS

5.1 Dataset statistics

When the full dataset is loaded, the home page reports the figures of Table 5.1. All values are calculated by the Java program from the CSV file; none is typed into the page.

Table 5.1 Dataset statistics shown by the system

Item	Value
Papers in the dataset	287,421
Research domains	146
Distinct author names	412,677
Distinct title words (vocabulary)	87,398
Average title length	9.6 words
Average abstract length	171 words

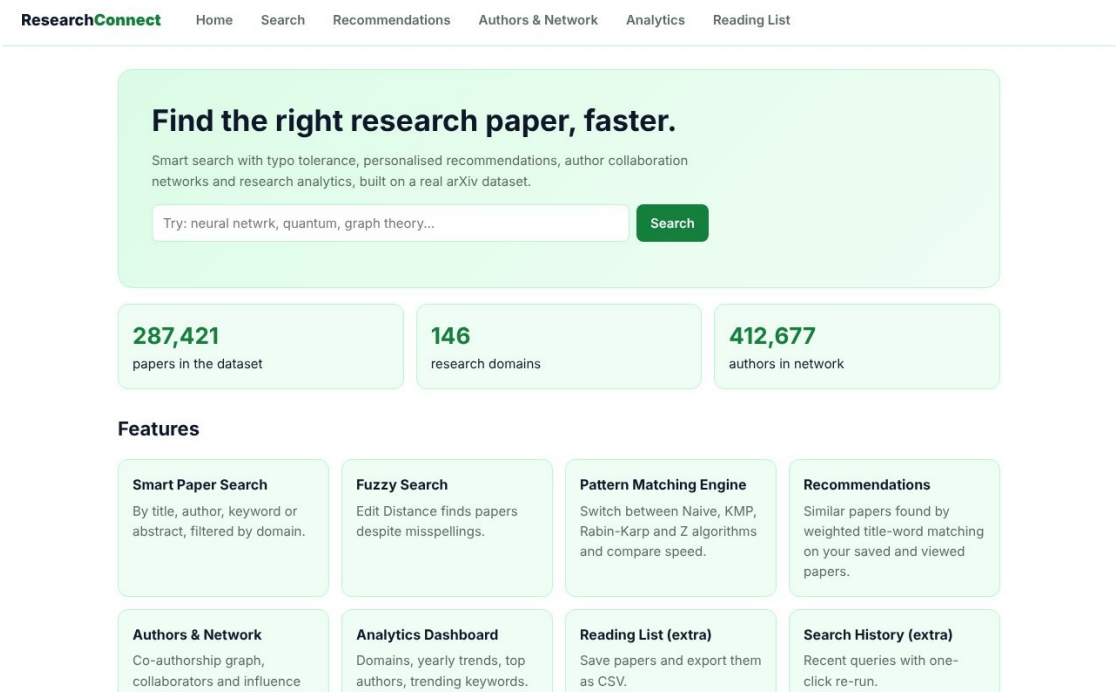


Fig. 5.1 Home page of Research Connect

The home page contains the top navigation bar, a search box, the dataset statistics and a summary of the features. The navigation bar stays at the top while the page is scrolled.

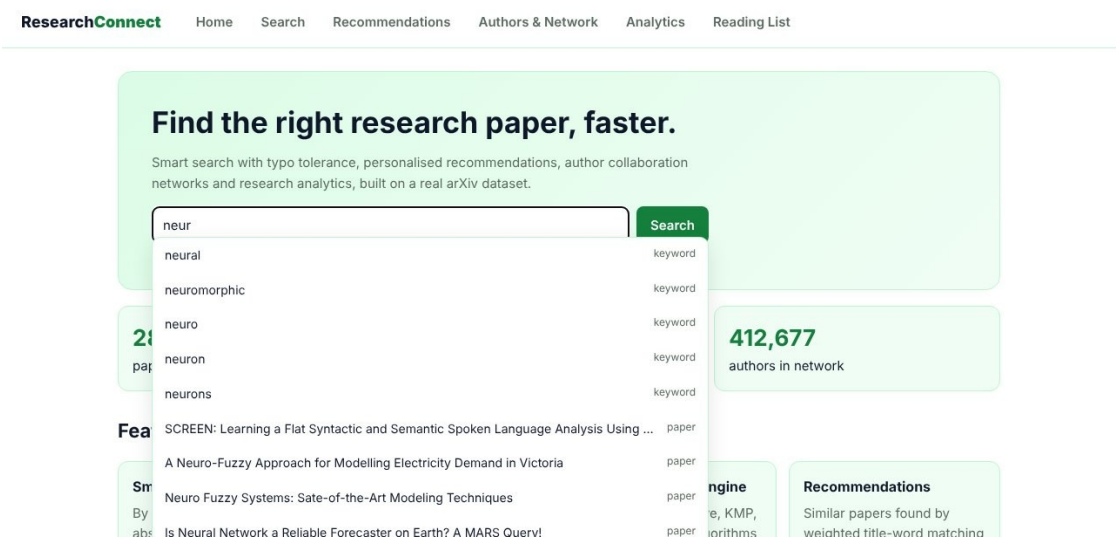


Fig. 5.2 Suggestions under the search box on the home page

While the user types, a dropdown shows matching keywords, paper titles and authors, each labelled with its kind. Choosing a suggestion opens the Search page with that text already searched.

5.2 Smart search

The Search page offers a field selector (all fields, title, author, abstract), a domain filter, the algorithm selector and a sort option. The result line shows the number of papers found, the algorithm used and the time taken in milliseconds. Each paper card shows the title, clickable authors, domain, year, id, a shortened abstract, and the Save and Similar papers buttons. Fig. 5.3 shows the search for "neural network" with KMP, which found 46,941 papers.

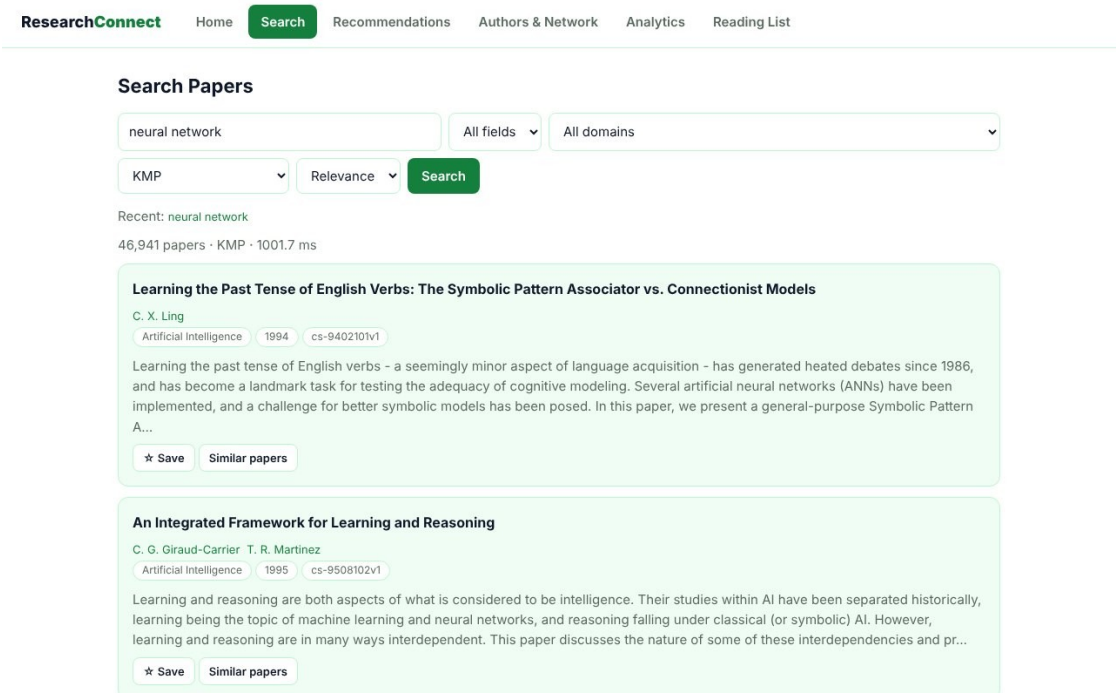


Fig. 5.3 Search results for "neural network" using KMP

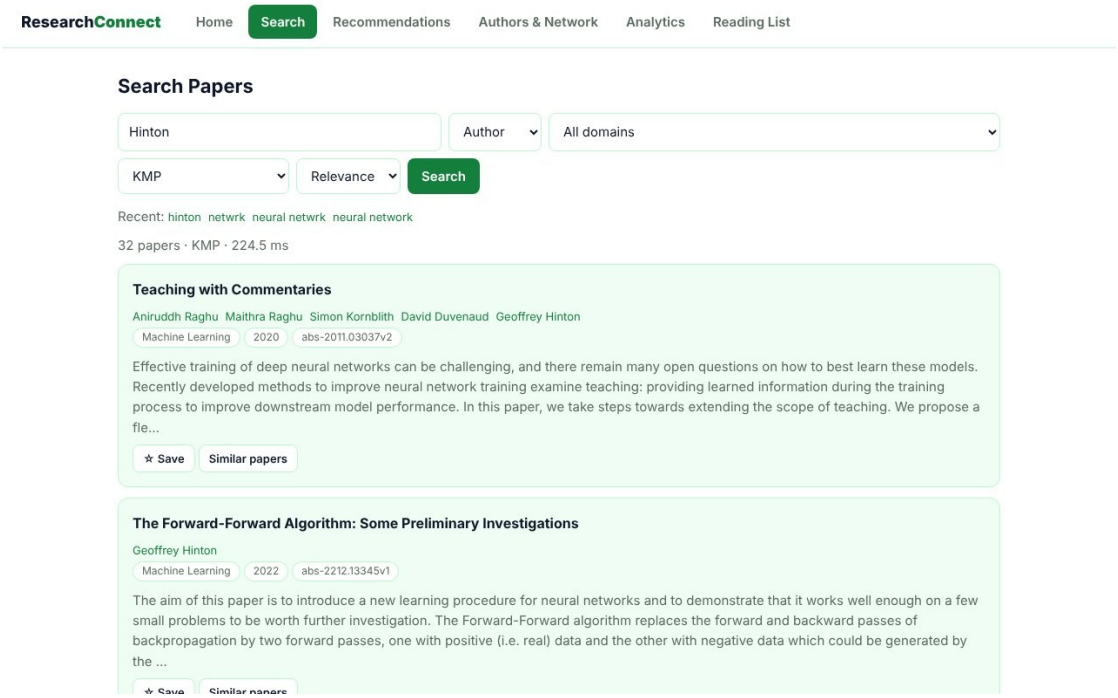


Fig. 5.4 Searching the author field for "Hinton"

5.3 Fuzzy search and "Did you mean"

With the Fuzzy (Edit Distance) option a misspelled query such as "neural netwrk" still returns papers about neural networks: 49,926 papers were found. When a normal search finds nothing, a "Did you mean" link with the closest vocabulary word is displayed; for "netwrk" the suggestion is "network".

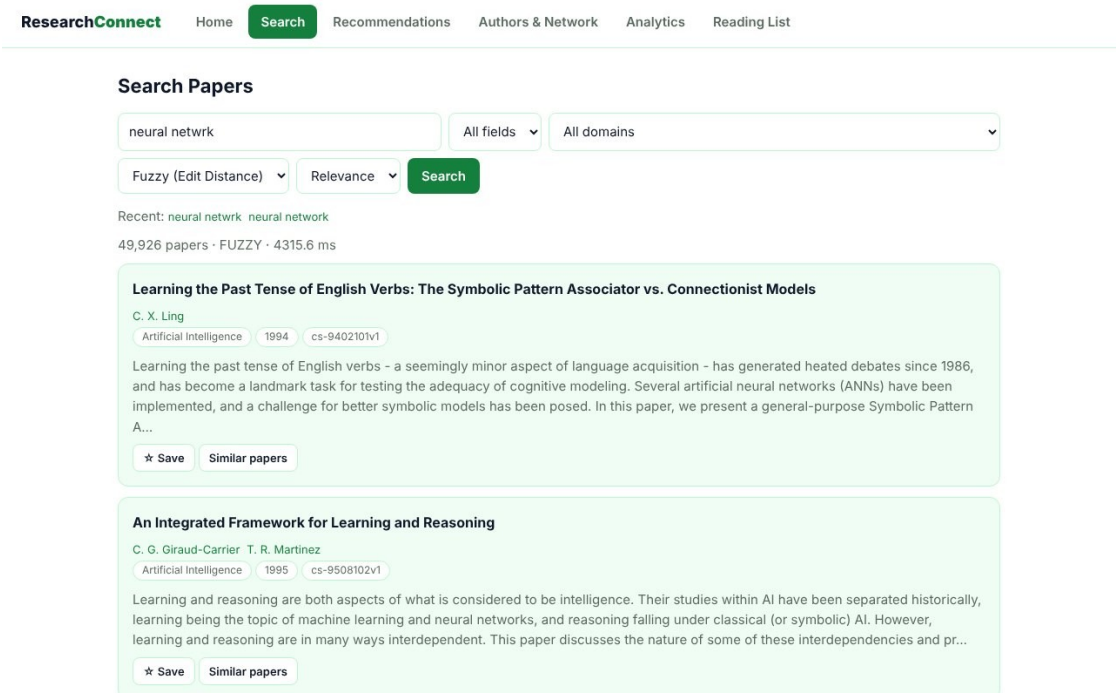


Fig. 5.5 Fuzzy search for the misspelled query "neural netwrk"

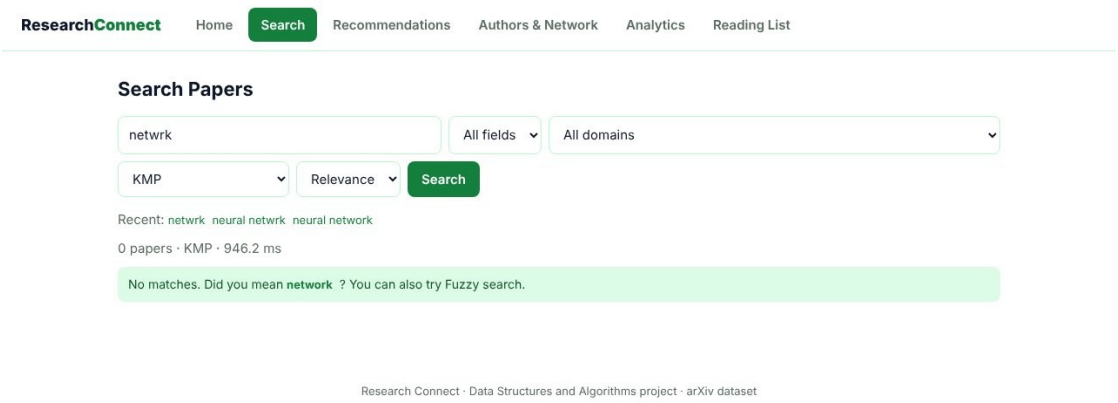


Fig. 5.6 "Did you mean" suggestion for "netwrk"

Table 5.2 Fuzzy search results on the full dataset

Misspelled query	Papers found	Time (ms)
neural netwrk	49,926	6,983
quantm computing	2,326	5,998
reinforcment learning	19,123	3,922

5.4 Recommendations

When the user saves papers or opens Similar papers, the recommendation module lists related papers with a match percentage. Fig. 5.7 shows the similar papers found for a saved paper about quantum computing, and Fig. 5.8 shows the Recommendations page built from two saved papers.

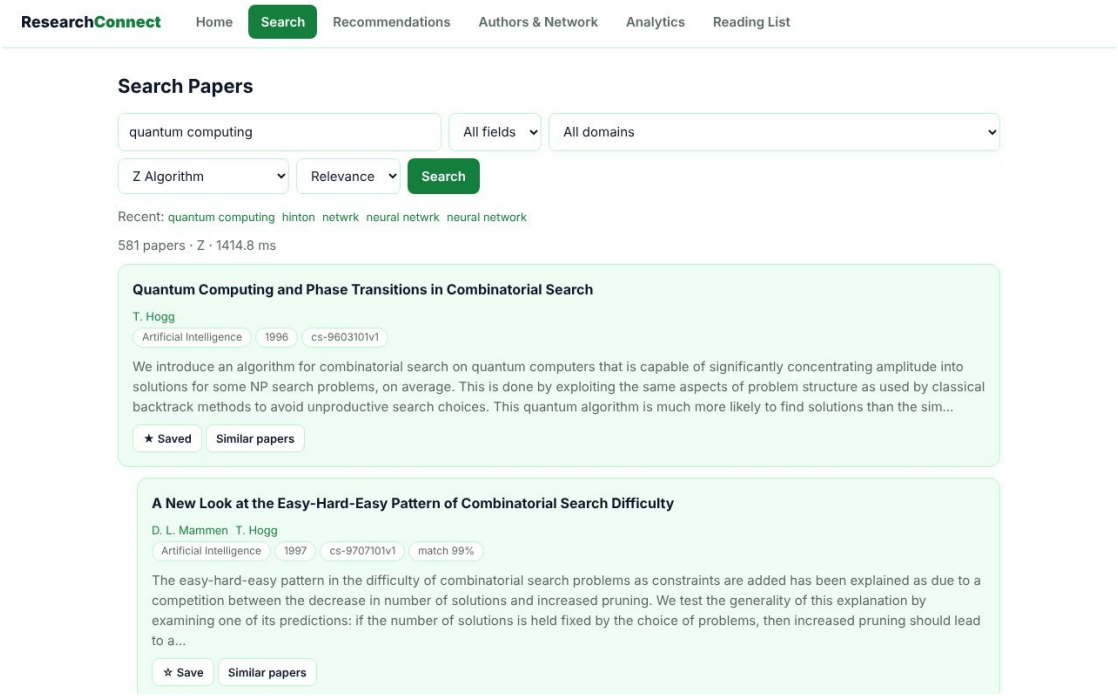


Fig. 5.7 Similar papers below a search result

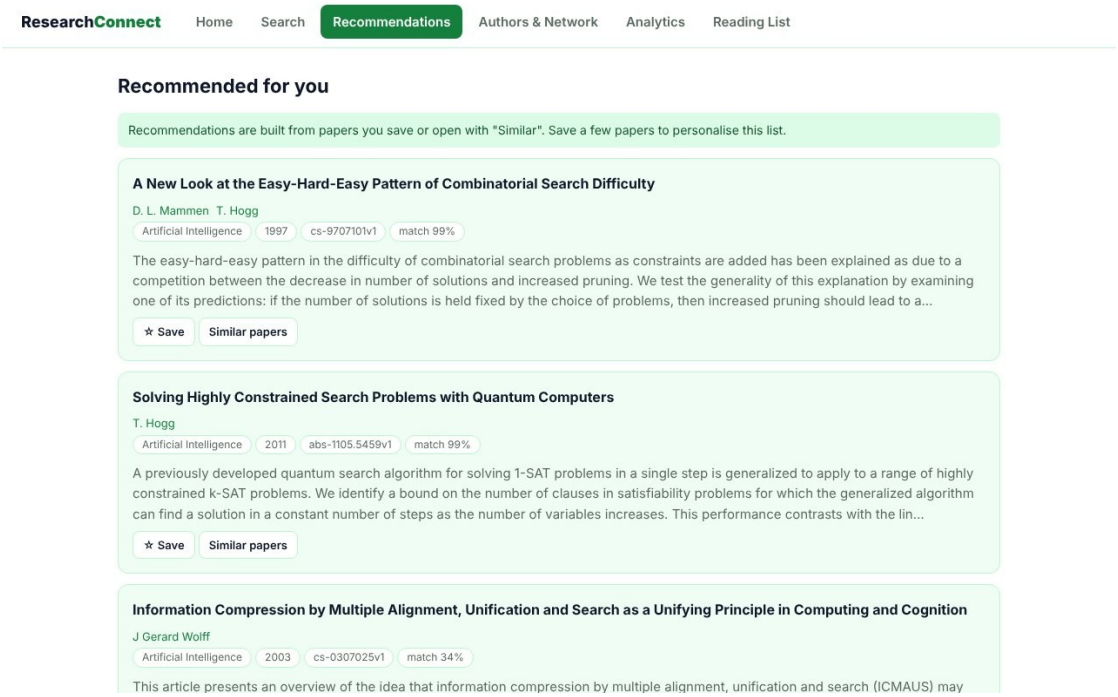


Fig. 5.8 Recommendations page

5.5 Authors and network

The Authors & Network page lists the most productive authors with the number of papers, the number of collaborators and the influence score. Selecting an author shows the papers and the clickable collaborators, so the user can move through the collaboration network from author to author.

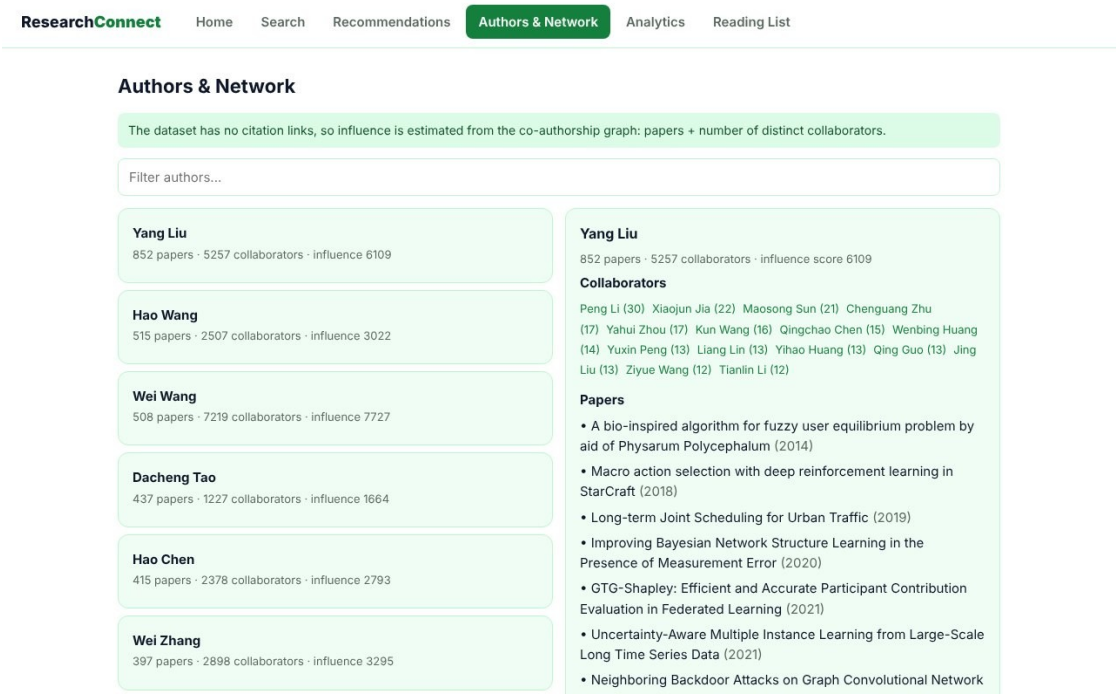


Fig. 5.9 Author details with collaborators and papers

Table 5.3 Top authors reported by the program

Author	Papers	Collaborators	Influence
Yang Liu	852	5,257	6,109
Hao Wang	515	2,507	3,022
Wei Wang	508	7,219	7,727
Dacheng Tao	437	1,227	1,664
Hao Chen	415	2,378	2,793
Wei Zhang	397	2,898	3,295
Xiang Li	385	2,268	2,653
Jun Wang	377	1,798	2,175
Bo Li	376	5,913	6,289
Xin Li	375	6,605	6,980

The dataset has no unique author identifiers, so different researchers with the same name, for example "Wei Wang", are counted together. This makes the numbers of very common names larger than the real values.

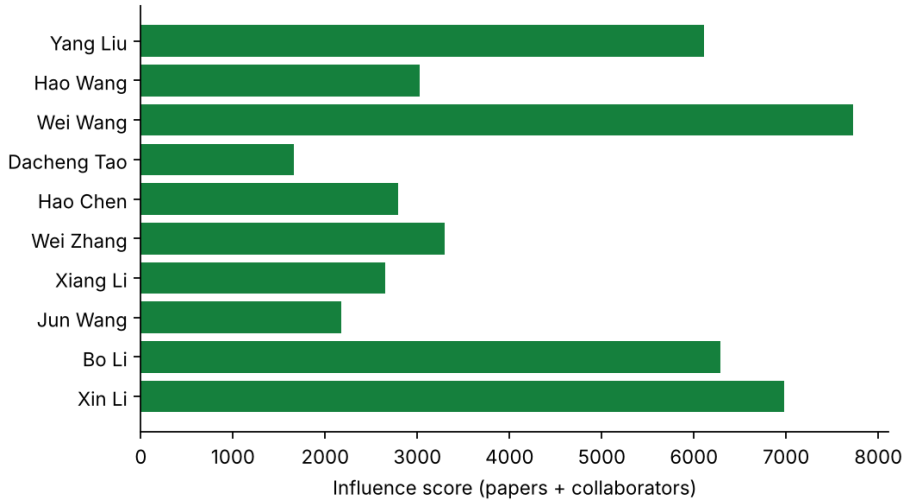


Fig. 5.10 Top ten authors by influence score

5.6 Reading list

Saved papers are listed on the Reading List page, which also shows them as CSV text (id, title, category, year) that can be copied into a spreadsheet or a reference list. The list is stored in data/saved.txt and is still present after the program is restarted.

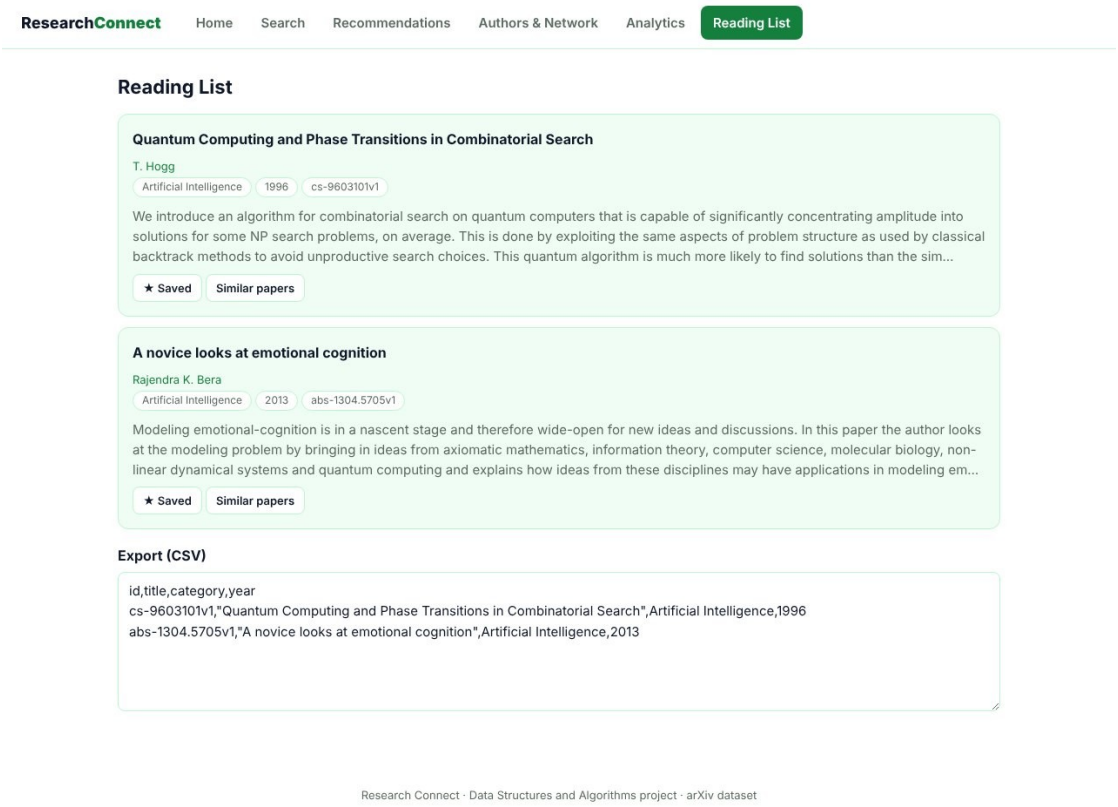


Fig. 5.11 Reading list with CSV text

5.7 Analytics dashboard

The dashboard shows four charts calculated from the whole dataset: papers by domain, top authors,

papers per year and trending keywords. Fig. 5.12 is a screenshot of the dashboard and the following figures present the same data in more detail.

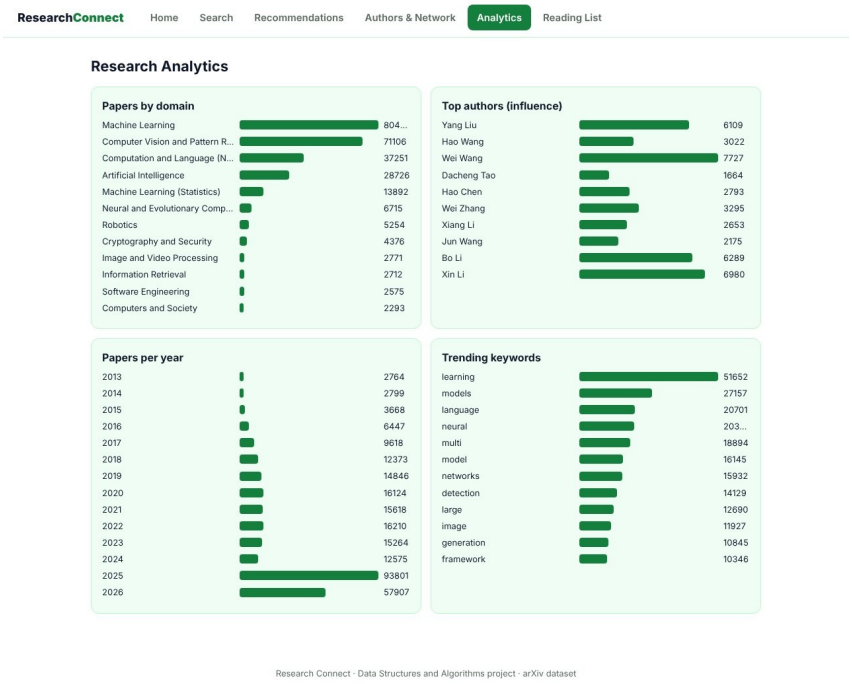


Fig. 5.12 Analytics dashboard

Table 5.4 Papers by domain (top 20)

Domain	Papers
Machine Learning	80,452
Computer Vision and Pattern Recognition	71,106
Computation and Language (Natural Language Processing)	37,251
Artificial Intelligence	28,726
Machine Learning (Statistics)	13,892
Neural and Evolutionary Computing	6,715
Robotics	5,254
Cryptography and Security	4,376
Image and Video Processing	2,771
Information Retrieval	2,712
Software Engineering	2,575
Computers and Society	2,293
Human-Computer Interaction	2,184
Sound	1,733
Methodology (Statistics)	1,616
Optimization and Control	1,554
Signal Processing	1,316
Distributed, Parallel, and Cluster Computing	1,133
Quantum Physics	1,052
Multiagent Systems	917

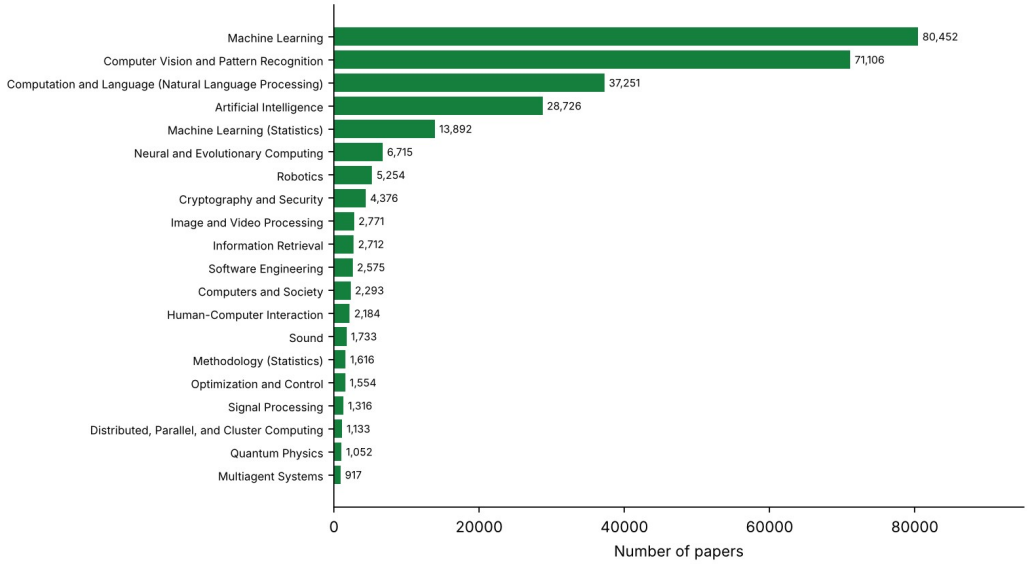


Fig. 5.13 Papers by domain (top 20)

Machine Learning is the largest domain with 80,452 papers, followed by Computer Vision and Pattern Recognition with 71,106 papers and Computation and Language with 37,251 papers. These three domains together hold more than half of all papers.

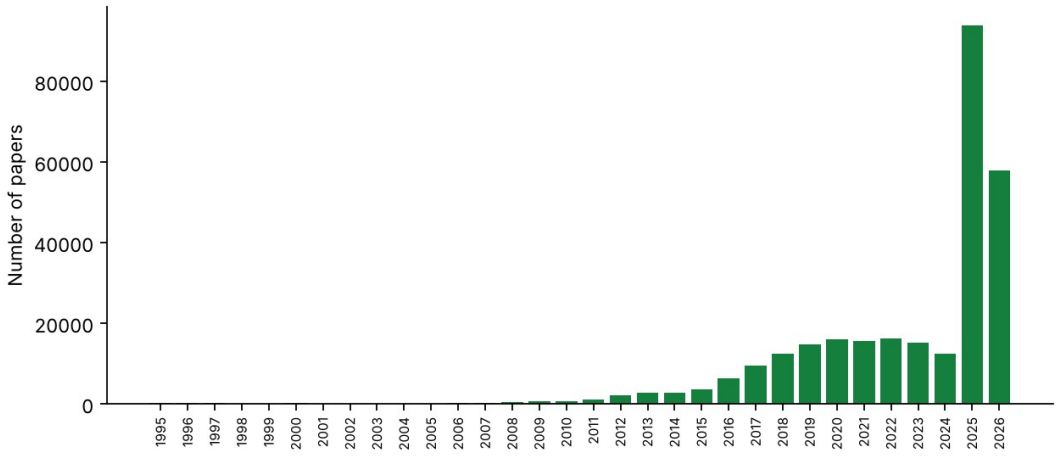


Fig. 5.14 Papers per year

The number of papers grows strongly after 2016. The dataset holds 93,801 papers for 2025 and 57,907 for 2026; the figure for 2026 is incomplete because that year was not finished when the dataset was collected, and the unusually large counts for 2025 and 2026 are a property of this dataset.

Table 5.5 Papers per year (selected years)

Year	Papers	Year	Papers
2013	2,764	2020	16,124
2014	2,799	2021	15,618
2015	3,668	2022	16,210

2016	6,447	2023	15,264
2017	9,618	2024	12,575
2018	12,373	2025	93,801
2019	14,846	2026	57,907

Table 5.6 Trending title keywords

Keyword	Titles	Keyword	Titles
learning	51,652	large	12,690
models	27,157	image	11,927
language	20,701	generation	10,845
neural	20,333	framework	10,346
multi	18,894	efficient	10,268
model	16,145	network	10,247
networks	15,932	graph	9,283
detection	14,129	aware	8,874

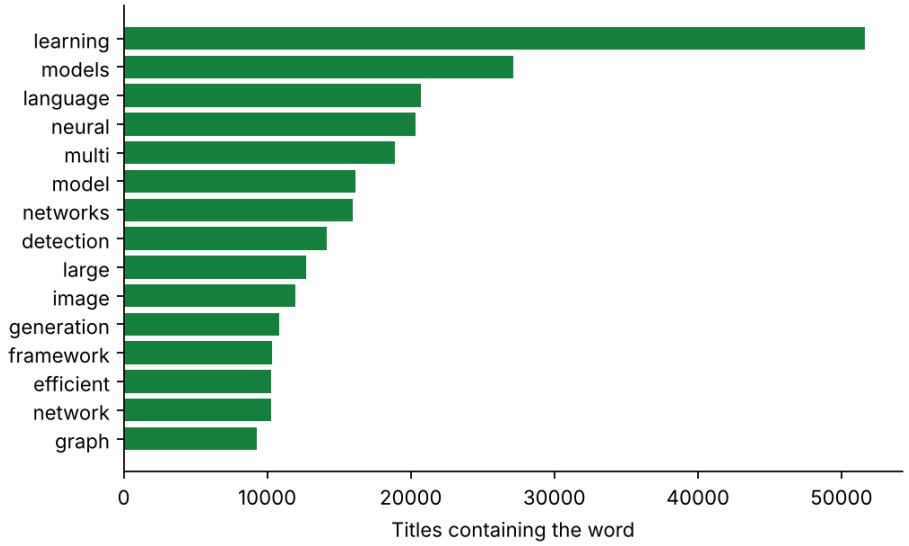


Fig. 5.15 Most frequent title keywords

The counts give the number of titles that contain the word, counted once per paper. Words such as "multi" appear because hyphenated words like "multi-agent" are split at the hyphen.

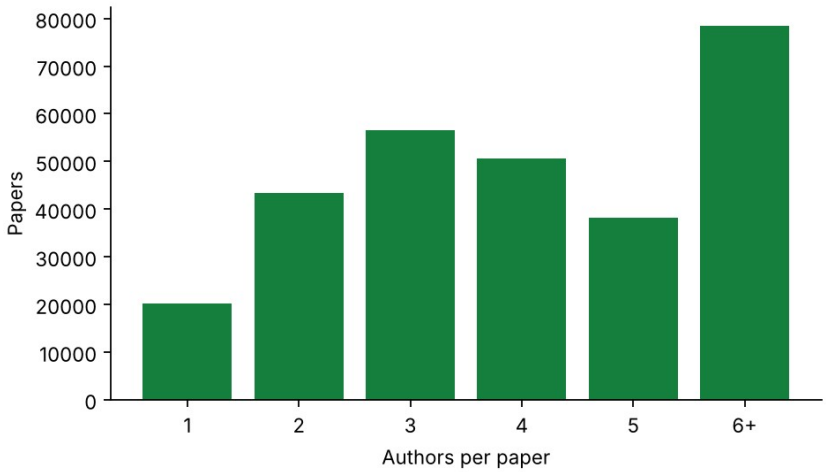


Fig. 5.16 Number of authors per paper

Most papers have several authors: 78,385 papers have six or more authors and only 20,181 papers have a single author. This is why the co-authorship network is dense and the collaborator counts of active authors are large.

5.8 Comparison of the algorithms

All four exact algorithms return the same papers for the same query; they differ only in speed. The Search page prints the time of every search, so the algorithms can be compared on the same query. Each query was run three times with each algorithm on the full dataset, in all fields, on a computer with one CPU core and 4 GB of RAM. Table 5.7 shows the best time and the average of the three runs.

Table 5.7 Search time of the algorithms (milliseconds)

Query	Papers found	Algorithm	Best of 3	Average of 3
neural network	46,941	Naive	1728	1809
neural network	46,941	KMP	1386	1458
neural network	46,941	Rabin-Karp	1980	2029
neural network	46,941	Z	1591	1764
quantum	2,484	Naive	1005	1081
quantum	2,484	KMP	807	996
quantum	2,484	Rabin-Karp	2096	2199
quantum	2,484	Z	1086	1130
reinforcement learning	19,105	Naive	1103	1201
reinforcement learning	19,105	KMP	1329	1357
reinforcement learning	19,105	Rabin-Karp	1866	1981
reinforcement learning	19,105	Z	1327	1412
graph	34,777	Naive	906	1005

graph	34,777	KMP	923	1036
graph	34,777	Rabin-Karp	1957	1977
graph	34,777	Z	1111	1154

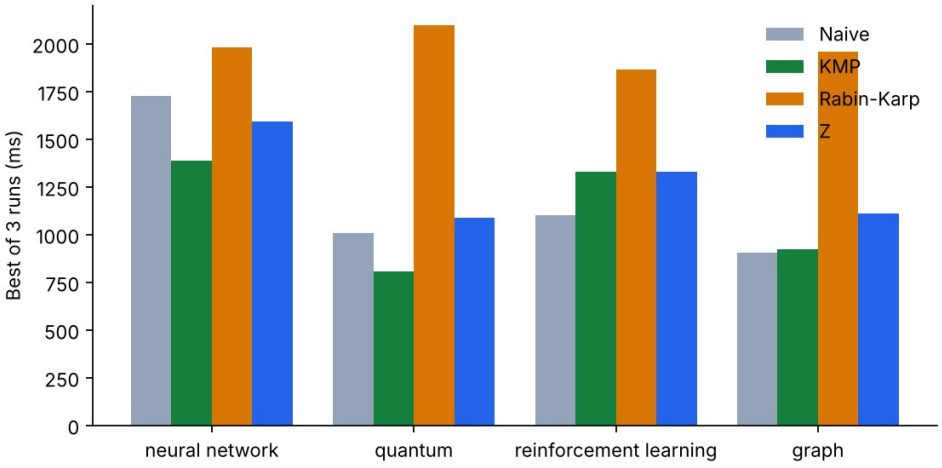


Fig. 5.17 Best search time of each algorithm

The observations are:

- The number of papers found is identical for all four algorithms, which confirms that they are correct and equivalent.
- KMP was the fastest or nearly the fastest for most queries, but the difference to the Naive search was small. Ordinary English text rarely produces the long partial matches that make the Naive search slow, so its worst case O(n x m) is not reached in practice.
- Rabin-Karp was the slowest in these measurements, roughly 1.5 to 2.5 times slower than KMP. It must calculate and update a hash with a modulus operation for every window of the text, and with short patterns it saves almost no comparisons.
- The Z algorithm was close to KMP but needed extra memory to build the combined string and the Z-array for every paper.
- Fuzzy search is much slower (4 to 7 seconds) because every query word is compared with the whole vocabulary of 87,398 words and then several alternative words are searched.

These times were measured on a single core. With several cores the parallel stream divides the papers between the cores and the exact searches become faster. The measurements show that for this application the choice of algorithm changes the speed only slightly, while the parallel scan and the in-memory data have a larger effect.

5.9 Discussion

The results show that the system meets its objectives: it searches the full arXiv dataset by several

fields, tolerates spelling mistakes, recommends related papers, shows author collaborations and presents research trends, all without a database. The parallel scan keeps exact searches responsive, while fuzzy search is slower because of the vocabulary comparison. The recommendation module gives sensible related papers because rare title words are weighted higher than common words, and the author network helps to move from one researcher to their collaborators.

The system also has limits, which are explained in Chapter 6: it has no real citation data, it merges authors with the same name, and it needs several gigabytes of memory.

5.10 Complexity and memory observations

The measured times agree with the complexity analysis of Table 3.4. The exact algorithms are linear in the amount of text, and the scan covers the title, the authors and the abstract of all 287,421 papers, which is a text of about 339 million characters, so a search takes about one to two seconds on one core. The cost of fuzzy search is dominated by the Edit Distance calculations: for a query of two words the program compares each word with the vocabulary of 87,398 words, and each comparison costs the product of the two word lengths.

The start-up time of about half a minute is spent reading the 430 MB file and building the maps. After that the memory use stays stable, because the search does not create copies of the papers; it only creates a list of the positions of the matching papers. The loader was run with a heap of 3 GB and did not run out of memory on the full dataset.

Table 5.8 Summary of the measured results

Measure	Result
Start-up (load of the full CSV)	about 25 to 34 seconds on a computer with 4 GB RAM and one core
Exact search of all papers	about 0.8 to 2.2 seconds per search on one core
Fuzzy search	about 4 to 7 seconds per search on one core
Results sent per request	15 papers
Distinct papers, domains, authors	287,421, 146, 412,677

5.11 Comparison with plain keyword search

A conventional search box matches only the exact text that the user types. Table 5.9 compares such a search with Research Connect for the situations described in the problem statement.

Table 5.9 Plain keyword search compared with Research Connect

Situation	Plain exact search	Research Connect
Correctly spelled query	Finds the papers	Finds the same papers (identical counts for all four algorithms)

Misspelled word ("netwrk")	No result	Offers "Did you mean network" and finds papers in fuzzy mode
Partial typing ("algor")	No result or a long wait	Suggests algorithm, algorithms and algorithmic while typing
Looking for related work	Not available	Recommendations from saved papers using rare shared words
Finding the people behind a topic	Not available	Author list with collaborators and influence score
Understanding trends	Not available	Charts of domains, years and keywords
Remembering interesting papers	Not available	Reading list stored in saved.txt

The comparison is qualitative. The project does not measure the relevance of the results against human judgements, which is a possible direction for future work.

5.12 Verification of the functional requirements

Table 5.10 Functional requirements and their verification

ID	Verified by
FR1	Tests T2, T7 and Figs. 5.3 and 5.4 (title, author and all-field search)
FR2	Test T6 (domain filter); the sort selector is on the Search page
FR3	Tests T2 and T3 (all four algorithms and the fuzzy mode)
FR4	Tests T4 and T5, Figs. 5.5 and 5.6
FR5	Test T8, Fig. 5.2
FR6	Test T9, Figs. 5.7 and 5.8
FR7	Test T10, Fig. 5.9
FR8	Section 5.7, Figs. 5.12 to 5.16
FR9	Test T11, Fig. 5.11
FR10	Recent searches line on the Search page, stored in history.txt
FR11	Result line of every search (count, algorithm and milliseconds)
FR12	15 papers per request and the Show more button

6. CHALLENGES & LIMITATIONS

6.1 Issues faced during development

- **Parsing the CSV:** abstracts contain commas, quotation marks and line breaks, so splitting the file by lines or commas broke the records.
- **Memory:** keeping 287,421 papers with their text in memory needs about 3 GB.
- **Speed of fuzzy search:** comparing every query word with the vocabulary and searching many alternatives was slow.
- **Showing large result sets:** thousands of matches cannot be sent to the browser at once.
- **Dates:** the publication date uses two-digit years, which cannot be read as a year directly.
- **Duplicate author names:** the dataset has no author identifiers, and an author listed twice in one paper was counted twice.
- **No citation data:** the dataset contains no citation links, so real citation analysis was not possible.
- **Browser events:** a click on the text inside a button did not trigger the button's action until the click handler was changed to look for the nearest element with the data attribute.

6.2 Solutions applied

- A quote-aware, character-by-character reader was written with BufferedReader.
- Only the first 400 characters of each abstract are stored for display, the full abstract is used for searching, and the heap size is set with -Xmx3g.
- Only the five most frequent alternatives per word are used in fuzzy search, and the scan runs in parallel.
- The server sends 15 results per request with an offset and a Show more button.
- Two-digit years above 26 are read as 19xx and the others as 20xx.
- The authors of one paper are put in a set before they are added to the author map.
- Influence is defined from the co-authorship graph (papers plus collaborators), and the interface says clearly that it is not based on citations.
- The click handler uses closest() to find the element that carries the data attribute.

6.3 Limitations of the System

- There are no real citation counts or citation links; influence is based on collaboration only.
- Authors with the same name are treated as one person.

- The whole dataset is held in memory, so the program needs several gigabytes of RAM and about half a minute to start.
- The recommendation index uses title words only and does not understand the meaning of the words.
- The match percentage in the recommendations is relative to the best result and is not a true similarity score.
- Fuzzy search over the whole dataset is slower than exact search.
- The program is meant for one user on a local computer and has no login system.
- Only graph degree is used for the network; there is no shortest-path or PageRank analysis.

7. CONCLUSION & FUTURE WORK

7.1 Conclusion

Research Connect is a research paper search and recommendation system that works on a real dataset of 287,421 arXiv papers. It combines the Naive, KMP, Rabin-Karp and Z string-matching algorithms with Edit Distance for typo-tolerant search, provides suggestions and recommendations, shows an author collaboration network with influence scores, and presents analytics about domains, years, authors and keywords. The data is managed using Java file handling and in-memory data structures instead of a database, and the user interface is a clean web page with navigation buttons at the top.

The measurements show that all four exact algorithms return identical results, that KMP and the Naive search are fastest on this kind of text, and that fuzzy search costs several seconds on the full dataset. The project shows how data structures and algorithms solve a practical information-retrieval problem and makes finding relevant academic literature faster and easier for students, researchers and academicians.

7.2 Future Work

- Add real citation data, for example from Semantic Scholar or OpenAlex, to compute citation counts and citation graphs.
- Use unique author identifiers such as ORCID to separate authors with the same name.
- Add graph algorithms such as breadth-first search, shortest path between authors and PageRank for a better influence measure.
- Rank results with TF-IDF or BM25 and extend the inverted index to the abstracts for faster search.
- Speed up fuzzy search with a BK-tree or a trie over the vocabulary.
- Add user accounts, personal profiles and a database or a Spring Boot service for use by many users.
- Add downloadable reports and charts and a mobile-friendly installable version.
- Integrate online academic databases and machine-learning based recommendations.

7.3 Skills learned during development

- Implementing KMP, Rabin-Karp, Z and Edit Distance and understanding their time complexity.

- Choosing suitable data structures (hash maps, inverted index, tree maps) for fast lookup.
- Java file handling with BufferedReader, FileReader, FileWriter and the Files class.
- Building a REST-style API with the JDK HTTP server and parallel streams.
- Creating a responsive front end with HTML, CSS and JavaScript and connecting it with fetch.
- Measuring and comparing the speed of algorithms on a large data set.
- Testing, debugging and documenting a project and working as a team.

REFERENCES

[1] D. E. Knuth, J. H. Morris and V. R. Pratt, “Fast pattern matching in strings,” SIAM Journal on Computing, vol. 6, no. 2, pp. 323–350, 1977.

[2] R. M. Karp and M. O. Rabin, “Efficient randomized pattern-matching algorithms,” IBM Journal of Research and Development, vol. 31, no. 2, pp. 249–260, 1987.

[3] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions and reversals,” Soviet Physics Doklady, vol. 10, no. 8, pp. 707–710, 1966.

[4] D. Gusfield, Algorithms on Strings, Trees, and Sequences. Cambridge University Press, 1997.

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022.

[6] C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval. Cambridge University Press, 2008.

[7] F. Ricci, L. Rokach and B. Shapira (Eds.), Recommender Systems Handbook, 3rd ed. Springer, 2022.

[8] M. E. J. Newman, “The structure of scientific collaboration networks,” Proceedings of the National Academy of Sciences, vol. 98, no. 2, pp. 404–409, 2001.

[9] arXiv.org e-print archive, Cornell University. https://arxiv.org

[10] Oracle, Java SE API documentation: package com.sun.net.httpserver. https://docs.oracle.com/en/java/javase/17/docs/api/jdk.httpserver/com/sun/net/httpserver/package-summary.html

[11] Oracle, The Java Tutorials: Basic I/O. https://docs.oracle.com/javase/tutorial/essential/io/

[12] Mozilla Developer Network, Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API